

Sveučilište Jurja Dobrile u Puli

Fakultet informatike u Puli

Zagrebačka 30, 52100 Pula, Hrvatska

Lorena Pavličić

Lucija Baljak

Kelvin Jurković

Vice Jukić

Sustav za upravljanje knjižnicom

Tim 11

Dokumentacija projekta iz kolegija Baze podataka I

Nositelj kolegija: doc. dr. sc. Goran Oreški

Izvođač: Romeo Šajina, mag. inf.

Pula, svibanj 2026.

Sadržaj

1. Uvod	1
2. Opis poslovnog procesa	2
3. Entity Relationship (ER) dijagram	2
4. Veze entiteta prema ER dijagramu	3
5. Sheme relacijskog modela	5
6. EER dijagram (MySQL Workbench)	5
7. SQL tablice	6
7.1. TABLICA autor	6
7.2. TABLICA izdavac	7
7.3. TABLICA zanr	7
7.5. TABLICA lokacija	9
7.6. TABLICA clan	9
7.7. TABLICA zaposlenik	10
7.8. TABLICA knjiga	11
7.9. TABLICA knjiga_autor	12
7.10. TABLICA knjiga_zanr	13
7.11. TABLICA primjerak	14
7.12. TABLICA rezervacija	15
7.13. TABLICA posudba	16
7.14. TABLICA razlog_kazne	18
7.15. TABLICA kazna	18
8. Pogledi i upiti	20
8.1 Lorena Pavličić	20
8.1.1 Pogled 1: v_bibliografski_katalog	20
8.1.2 Pogled 2: v_broj_knjiga_po_autoru	22
8.1.3 Pogled 3: v_broj_autora_po_knjizi	23
8.1.4 Upit 1: Prikazuje sve knjige određenog autora	24
8.1.5 Upit 2: Prikazuje autore koji imaju dvije ili više knjiga u katalogu.	25
8.1.6 Upit 3: Prikazuje knjige koje imaju više od jednog autora.	27
8.2 Lucija Baljak	28
8.2.1 Pogled 1: ocekivane_kazne	28
8.2.2 Select 1: Pregled kašnjenja po članu	30
8.2.3 Pogled 2: zaposlenici_statistika	31
8.2.4 Select 2: Učinkovitost zaposlenika — godišnji izvještaj za 2025.	33
8.2.5 Pogled 3: kazne_pregled	35
8.2.6 Select 3: Kronološki trend kazni po mjesecima	36

8.3 Kelvin Jurković	38
8.3.1 Pogled 1: statistika_clanova	38
8.3.2 Pogled 2: aktivne_posudbe_detalji	40
8.3.3 Pogled 3: rezervacije_dostupnost	43
8.3.4 Upit 1: Članovi s iznadprosječnim brojem aktivnih posudbi	46
8.3.5 Upit 2: Aktivne posudbe koje kasne ili uskoro ističu	48
8.3.6 Upit 3: Knjige s više aktivnih rezervacija nego dostupnih primjeraka	50
8.4 Vice Jukić	53
8.4.1. Pogled 1: analiza_nedostupnih_primjeraka	53
8.4.2. Upit 1: Analiza odjela s povećanim brojem nedostupnih primjeraka	54
8.4.3. Pogled 2: fond_po_odjelu	56
8.4.4. Upit 2: Analiza najzastupljenijih odjela knjižničkog fonda	57
8.4.5. Pogled 3: analiza_fonda_po_zanru	58
8.4.6. Upit 3: Analiza žanrova s natprosječnim brojem primjeraka	60

1. Uvod

Naš projekt razvijao se postupno kroz izradu relacijskog modela, SQL skripti, testnih podataka, ER dijagrama i EER dijagrama. Tema projekta je Sustav za upravljanje knjižnicom, a cilj je bio osmisliti bazu podataka koja može podržati osnovne poslovne procese knjižnice, kao što su evidencija knjiga, autora, izdavača, žanrova, fizičkih primjeraka, članova, zaposlenika, posudbi, rezervacija i kazni. Tijekom rada na projektu bolje smo razumjeli način na koji se stvarni poslovni proces može pretvoriti u relacijski model baze podataka. Posebnu pažnju posvetili smo pravilnom povezivanju relacija, određivanju primarnih i stranih ključeva, modeliranju veza različitih kardinalnosti te izradi složenih SQL upita i pogleda koji imaju praktičnu svrhu u radu knjižnice. Za razvoj projekta koristili smo GitHub za verzioniranje i dijeljenje koda, Visual Studio Code i MySQL Workbench za pisanje i testiranje SQL skripti, a za izradu dijagrama koristili smo alate za modeliranje baze podataka. ER dijagram izrađen je kako bi se prikazao konceptualni model sustava, dok je EER dijagram generiran u MySQL Workbenchu pomoću opcije reverse engineering na temelju postojeće SQL sheme. Baza podataka sastoji se od više povezanih relacija koje zajedno opisuju rad knjižnice. U sustavu se pohranjuju podaci o knjigama, autorima, izdavačima, žanrovima i fizičkim primjercima knjiga, ali i podaci o članovima knjižnice, zaposlenicima, posudbama, rezervacijama i kaznama. Time sustav ne služi samo kao katalog knjiga, nego omogućuje i praćenje dostupnosti primjeraka, aktivnosti članova, najčešće posuđivanih knjiga, rezervacija i zakašnjelih povrata. Podaci korišteni u projektu uneseni su kao testni podaci i služe isključivo za demonstraciju rada baze podataka. Osobni podaci članova i zaposlenika nisu stvarni, već su izmišljeni ili pseudo-nasumično generirani. Moguće sličnosti sa stvarnim osobama ili podacima potpuno su slučajne. Projekt je podijeljen na nekoliko funkcionalnih cjelina. Bibliografski katalog obuhvaća podatke o knjigama, autorima i izdavačima. Klasifikacija i fond opisuju žanrove, lokacije i fizičke primjerke knjiga. Modul članova, posudbi i rezervacija prikazuje kako članovi koriste knjižničnu građu, dok modul zaposlenika i kazni prati rad zaposlenika te obračun kazni za zakašnjele povrate. U nastavku dokumentacije prikazani su ER i EER dijagrami, opisane su sve relacije, atributi i domene, objašnjena su poslovna pravila i ograničenja baze podataka te su

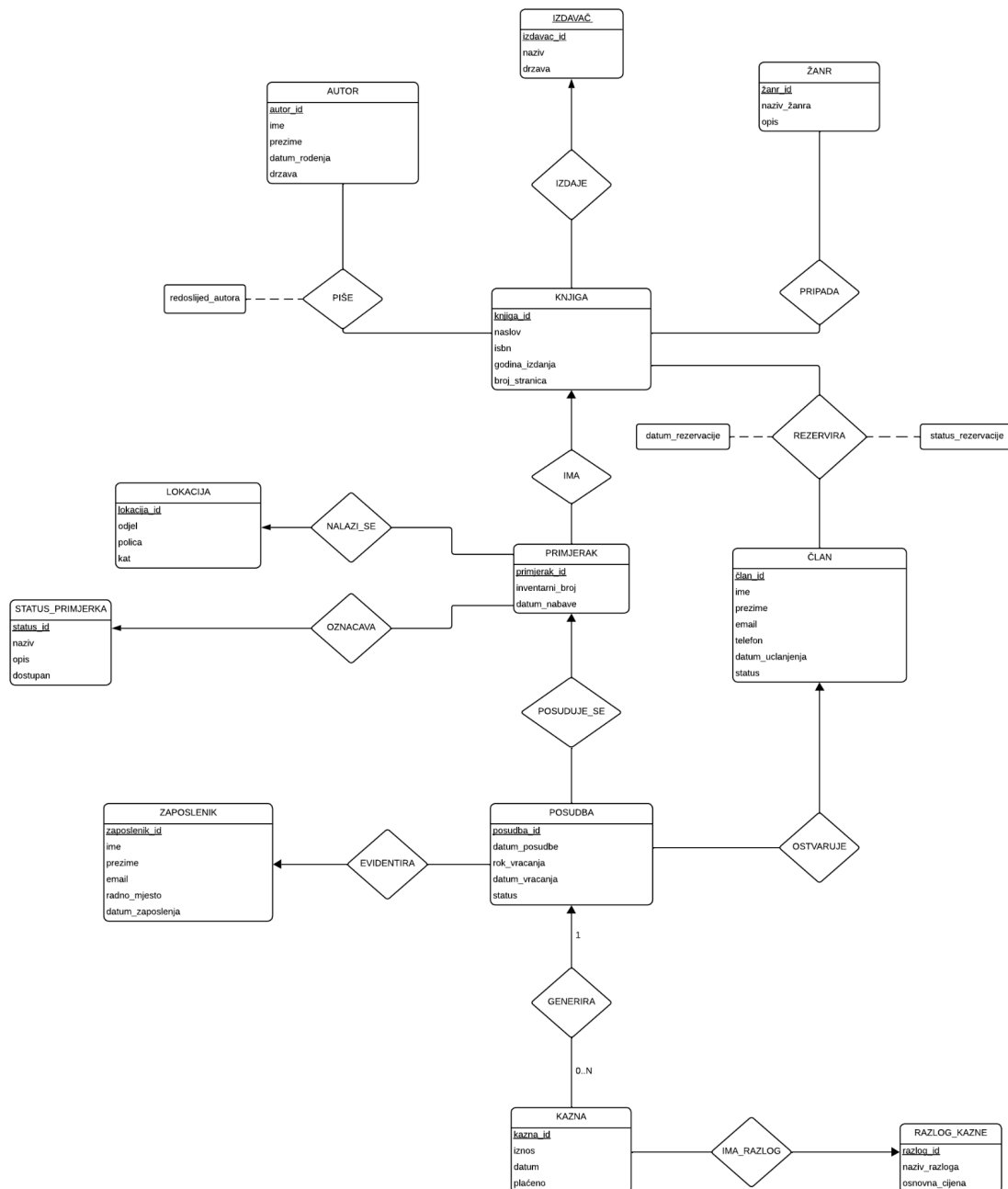
prikazani izrađeni pogledi i složeni SQL upiti. Na kraju je dan zaključak o funkcionalnosti sustava i mogućnostima daljnjeg proširenja baze podataka.

2. Opis poslovnog procesa

U knjižnici se prati cjelokupan ciklus posuđivanja knjiga. Za svakog člana knjižnice vode se sljedeći podaci: ime, prezime, e-mail, telefon, datum učlanjenja i status: Aktivan, Blokirani ili Neaktivan. Knjižnica zapošljava zaposlenike na različitim radnim mjestima: knjižničar, voditelj odjela, administrator, arhivar, voditelj nabave, pomoćni knjižničar i stručni suradnik. Za svakog zaposlenika prate se ime, prezime, e-mail, radno mjesto i datum zaposlenja. Knjižnični fond čine knjige za koje se evidentiraju naslov, ISBN, godina izdanja i broj stranica. Svaka knjiga povezana je s jednim izdavačem, a može imati više autora, uz definirani redoslijed autorstva, te pripadati jednom ili više žanrova. Za autora se prate ime, prezime, datum rođenja i država, a za izdavača naziv i država. Fizički primjerci knjiga vode se odvojeno od bibliografskog zapisa. Svaki primjerak ima jedinstveni inventarni broj, datum nabave, lokaciju u knjižnici — odjel, policu i kat — te status: Dostupno, Posuđeno, Rezervirano, Oštećeno, Izgubljeno, U obradi, Na popravku itd. Član može rezervirati knjigu prije nego što postane dostupna, a rezervacija ima status: Aktivna, Istekla ili Otkazana. Posudba se evidentira kada zaposlenik izda konkretan primjerak članu. Bilježe se datum posudbe, rok vraćanja, datum vraćanja, koji može biti NULL ako je posudba aktivna, i status posudbe. Ako član vrati knjigu sa zakašnjenjem, oštetiti je ili izgubi, generira se kazna. Za svaku kaznu evidentiraju se razlog — Kašnjenje, Oštećenje ili Gubitak — iznos, datum i podatak o tome je li kazna plaćena. Cjenik osnovnih kazni vodi se u zasebnoj tablici razlog_kazne: kašnjenje iznosi 0,50 EUR po danu, oštećenje 5,00 EUR, a gubitak 20,00 EUR. Jedna posudba može imati više kazni iz različitih razloga, primjerice istovremeno kašnjenje i oštećenje.

3. Entity Relationship (ER) dijagram

Sljedeći ER dijagram detaljno opisuje sve skupove entiteta, njihove atribute te skupove veza između njih. Kardinalnosti mapiranja označavaju koliko drugih entiteta može biti povezano s pojedinim entitetom putem određene veze.



4. Veze entiteta prema ER dijagramu

- IZDAVAČ izdaje KNJIGU (jedan izdavač može izdati više knjiga, dok je svaka knjiga izdana od strane točno jednog izdavača) — kardinalnost one to many
- AUTOR piše KNJIGU (jedan autor može napisati više knjiga, a jedna knjiga može biti napisana od više autora). Ovo je veza tipa many to many koja u relacijskom modelu zahtijeva veznu tablicu knjiga_autor. Na samoj vezi nalazi se atribut redoslijed_autora koji označava redoslijed autorstva (1 = prvi autor, 2 = drugi autor itd.)
- KNJIGA pripada ŽANRU (jedna knjiga može pripadati u više žanrova, npr. „1984" je i roman i znanstvena fantastika; jedan žanr obuhvaća više knjiga) — many to many, realizirano veznom tablicom knjiga_zanr
- KNJIGA ima PRIMJERAK (jedna knjiga može imati više fizičkih primjeraka u fondu, dok svaki primjerak pripada točno jednoj knjizi) — one to many
- PRIMJERAK se nalazi_se na LOKACIJI (svaki primjerak smješten je na točno jednoj lokaciji unutar knjižnice, dok jedna lokacija — odjel/polica/kat — može sadržavati više primjeraka) — one to many
- STATUS_PRIMJERKA označava PRIMJERAK (svaki primjerak je u točno jednom statusu — npr. Dostupno, Posuđeno, Oštećeno; isti status može opisivati više primjeraka) — one to many
- ČLAN rezervira KNJIGU (jedan član može rezervirati više knjiga, a jedna knjiga može biti rezervirana od više članova). Ovo je veza tipa many to many s vlastitim atributima datum_rezervacije i status_rezervacije, koja se u relacijskom modelu realizira zasebnom tablicom rezervacija. Bitno je primijetiti da rezervacija ide na razini knjige (apstraktnog naslova), a ne primjerka
- ČLAN ostvaruje POSUDBU (jedan član može tijekom vremena ostvariti više posudbi, dok svaka posudba pripada točno jednom članu) — one to many
- PRIMJERAK se posuđuje_se kroz POSUDBU (isti primjerak može biti posuđen više puta tijekom svog životnog vijeka, dok se jedna posudba odnosi na točno jedan primjerak) — one to many

- ZAPOSLENIK evidentira POSUDBU (jedan zaposlenik može evidentirati više posudbi, dok je svaka posudba realizirana od strane točno jednog zaposlenika) — one to many
- POSUDBA generira KAZNU (jedna posudba može generirati nula, jednu ili više kazni — kardinalnost $1 \rightarrow 0..N$). Posudba koja je vraćena na vrijeme i bez štete nema kaznu, dok posudba koja kasni i pritom je primjerak oštećen može imati dvije kazne istovremeno (jedna za kašnjenje, jedna za oštećenje) — one to many
- KAZNA ima_razlog iz RAZLOG_KAZNE (svaka kazna ima točno jedan razlog iz šifrnika razloga — Kašnjenje, Oštećenje ili Gubitak; isti razlog može biti vezan uz mnogo kazni) — one to many

5. Sheme relacijskog modela

autor (id_autor, ime, prezime, datum_rođenja, drzava)

izdavac (id_izdavac, naziv, drzava)

zanr (id_zanr, naziv, opis)

status_primjerka (id_status, naziv, opis, dostupan)

lokacija (id_lokacija, odjel, polica, kat)

clan (id_clan, ime, prezime, email, telefon, datum_uclanjenja, status)

zaposlenik (id_zaposlenik, ime, prezime, email, radno_mjesto, datum_zaposlenja)

knjiga (id_knjiga, id_izdavac, naslov, isbn, godina_izdanja, broj_stranica)

knjiga_autor (id_knjiga, id_autor, redoslijed_autora)

knjiga_zanr (id_knjiga, id_zanr)

primjerak (id_primjerak, id_knjiga, id_lokacija, id_status, inventarni_broj, datum_nabave)

rezervacija (id_rezervacija, id_clan, id_knjiga, datum_rezervacije, status_rezervacije)

posudba (id_posudba, id_clan, id_primjerak, id_zaposlenik, datum_posudbe, rok_vracanja, datum_vracanja, status)

razlog_kazne (id_razlog, naziv_razloga, osnovna_cijena)

kazna (id_kazna, id_posudba, id_razlog, iznos, datum, placeno)

6. EER dijagram (MySQL Workbench)

EER dijagram generiran je u MySQL Workbenchu pomoću opcije Reverse Engineering iz fizičke sheme baze podataka.

7. SQL tablice

7.1. TABLICA autor

Tablica autor služi evidenciji autora knjiga koje čine knjižnični fond. Sadrži attribute: id_autor, ime, prezime, datum_rođenja i drzava. Atribut id_autor je PRIMARY KEY tablice tipa INT jer se u njega unosi brojčana vrijednost. Služi kako bismo mogli jedinstveno označiti svakog autora u knjižnici. Iako atribut id_autor mora biti jedinstven, nije potrebno postaviti ograničenje UNIQUE jer je PRIMARY KEY jedinstven sam po sebi. Uz njega smo postavili AUTO_INCREMENT što znači da baza sama dodjeljuje sljedeću slobodnu vrijednost prilikom unosa novog autora, čime se izbjegava potreba za ručnim određivanjem id-ja. Atribut ime tipa je VARCHAR(50) koji nam omogućuje dinamičku alokaciju memorije. Odabrali smo tip VARCHAR s obzirom da ne znamo unaprijed koliko će znakova pojedino ime sadržavati. Maksimalna duljina definirana je u zagradi nakon tipa podatka, a iznosi 50 znakova što je dovoljno i za višestruka ili dvostruka imena. Postavljeno je ograničenje NOT NULL koje ne dopušta unos podataka bez ovog podatka, budući da autor bez imena ne bi imao smisla. Atribut prezime također je tipa VARCHAR(50) s istim NOT NULL ograničenjem, iz istih razloga kao i ime. Atribut datum_rođenja je tipa DATE jer označava datum kao cjelinu (bez vremena), a ovaj tip omogućuje lakše uspoređivanje i računanje s datumima u upitima. Za ovaj atribut NIJE postavljen NOT NULL jer za neke povijesne ili anonimne autore točan datum rođenja nije poznat. Posljednji atribut drzava je tipa VARCHAR(50) i također dopušta NULL vrijednost.

CREATE TABLE autor (

id_autor INT PRIMARY KEY AUTO_INCREMENT,

ime VARCHAR(50) NOT NULL,

```
prezime VARCHAR(50) NOT NULL,  
  
datum_rodenja DATE,  
  
drzava VARCHAR(50)  
  
);
```

7.2. TABLICA izdavac

Tablica izdavac služi evidenciji izdavačkih kuća s kojima knjižnica surađuje. Sadrži attribute: id_izdavac, naziv i drzava. Atribut id_izdavac je tipa INT jer u njega unosimo brojčanu vrijednost koja jedinstveno definira svakog pojedinog izdavača, što ga ujedno čini odličnim PRIMARY KEY atributom. Dodali smo AUTO_INCREMENT čime baza sama generira nove identifikatore. Atribut naziv označava naziv izdavačke kuće, tipa je VARCHAR(100) što je dovoljno za duga puna imena izdavača. Odabrali smo tip VARCHAR umjesto CHAR jer duljine naziva značajno variraju (od 'Znanje' do 'Oxford University Press'), pa je dinamička alokacija učinkovitija. Ograničenje NOT NULL garantira da izdavač mora imati naziv. Atribut drzava je tipa VARCHAR(50) i može biti NULL u slučajevima kada nemamo tu informaciju.

CREATE TABLE izdavac (

```
id_izdavac INT PRIMARY KEY AUTO_INCREMENT,  
  
naziv VARCHAR(100) NOT NULL,  
  
drzava VARCHAR(50)  
  
);
```

7.3. TABLICA zanr

Tablica zanr je referentna tablica (šifarnik) žanrova kojima knjiga može pripadati. Sadrži attribute: id_zanr, naziv_zanra i opis. Atribut id_zanr je PRIMARY KEY tipa INT s AUTO_INCREMENT, a označava brojčanu jedinstvenu vrijednost svakog žanra. Atribut naziv_zanra je tipa VARCHAR(50) jer ne znamo unaprijed kolika će biti duljina naziva (od kratkog 'Roman' do dugog 'Kriminalistički roman'). Na ovaj atribut postavljena su dva ograničenja: NOT NULL koje garantira obavezan unos i UNIQUE koje osigurava da dva žanra ne mogu imati isti naziv — 'Roman' može postojati samo

jednom u tablici. Atribut opis je tipa VARCHAR(255) i služi za kratko tekstualno objašnjenje žanra (npr. 'Knjige povijesne tematike'); ovaj atribut je opcionalan i može biti NULL.

```
CREATE TABLE zanr (  
  
    id_zanr INT PRIMARY KEY AUTO_INCREMENT,  
  
    naziv_zanra VARCHAR(50) NOT NULL UNIQUE,  
  
    opis VARCHAR(255)  
  
);
```

7.4. TABLICA *status_primjerka*

Tablica *status_primjerka* služi kao šifarnik mogućih statusa u kojima se može nalaziti fizički primjerak knjige. Sadrži attribute *id_status*, *naziv*, *opis* i *dostupan*.

Atribut *id_status* je PRIMARY KEY tipa INT s AUTO_INCREMENT.

Atribut *naziv* je tipa VARCHAR(30) te ima ograničenja NOT NULL i UNIQUE jer svaki status mora imati jedinstven naziv. Primjeri statusa su: 'Dostupno', 'Posudeno', 'Rezervirano', 'Osteceno', 'Izgubljeno', 'U obradi', 'Na popravku', itd.

Atribut *opis* je tipa VARCHAR(255) i služi za dodatno objašnjenje statusa primjerka.

Atribut *dostupan* je tipa BOOLEAN i označava je li primjerak trenutno dostupan korisnicima knjižnice ili nije. Ovo omogućuje jednostavnije filtriranje dostupnih i nedostupnih primjeraka unutar upita i pregleda.

Modeliranje statusa kao zasebne tablice predstavlja primjer dobre normalizacije baze podataka jer omogućuje jednostavno dodavanje novih statusa bez izmjene strukture tablice primjerak te sprječava dupliciranje i tipfelere u nazivima statusa.

```
CREATE TABLE status_primjerka (  
  
    id_status INT PRIMARY KEY AUTO_INCREMENT,  
  
    naziv VARCHAR(30) NOT NULL UNIQUE,  
  
    opis VARCHAR(255),
```

dostupan BOOLEAN NOT NULL

);

7.5. TABLICA lokacija

Tablica lokacija opisuje fizički smještaj primjeraka unutar knjižnice. Sadrži attribute: id_lokacija, odjel, polica i kat. Atribut id_lokacija je PRIMARY KEY tipa INT s AUTO_INCREMENT. Atribut odjel je tipa VARCHAR(50) i opisuje tematsku zonu knjižnice (npr. 'Beletristika', 'Strucna literatura', 'Djecji odjel'); obavezan je (NOT NULL) jer svaka lokacija mora pripadati nekom odjelu. Atribut polica je tipa VARCHAR(20) — kraća duljina od odjela jer su oznake polica obično kratke ('A1', 'B2', 'CIT1') i također je obavezan. Atribut kat je tipa VARCHAR(20) jer ga želimo prikazati kao tekst ('Prizemlje', '1. kat', '2. kat') a ne kao broj; ovaj atribut može biti NULL ako lokacija nije vezana uz konkretan kat.

CREATE TABLE lokacija (

id_lokacija INT PRIMARY KEY AUTO_INCREMENT,

odjel VARCHAR(50) NOT NULL,

polica VARCHAR(20) NOT NULL,

kat VARCHAR(20)

);

7.6. TABLICA clan

Tablica clan evidentira sve članove knjižnice. Sastoji se od atributa id_clan, ime, prezime, email, telefon, datum_uclanjenja i status. Atribut id_clan je PRIMARY KEY tipa INT s AUTO_INCREMENT. Atributi ime i prezime su tipa VARCHAR(50) s NOT NULL ograničenjem jer član mora imati identitet. Atribut email je tipa VARCHAR(100) — veća duljina nego za imena jer e-mail adrese mogu biti dugačke (lokalni dio + domena). Na ovaj atribut postavljeno je NOT NULL i UNIQUE: e-mail mora postojati jer se koristi za komunikaciju s članom, a UNIQUE garantira da svaka osoba ima samo jedan račun u sustavu. Atribut telefon je tipa VARCHAR(30) —

odabrali smo VARCHAR a ne INT iz dva razloga: prvo, brojevi telefona mogu početi nulom koju INT ne bi sačuvao, i drugo, mogu sadržavati formatne znakove ('+', '/', razmake). Telefon je opcionalan (može biti NULL) jer neki članovi možda ne žele dati broj. Atribut datum_uclanjenja je tipa DATE i obavezan je — knjižnica uvijek bilježi kad je član postao član. Atribut status je tipa VARCHAR(30) i obavezan; sadrži vrijednosti poput 'Aktivan', 'Blokiran' ili 'Neaktivan'.

CREATE TABLE clan (

id_clan INT PRIMARY KEY AUTO_INCREMENT,

ime VARCHAR(50) NOT NULL,

prezime VARCHAR(50) NOT NULL,

email VARCHAR(100) NOT NULL UNIQUE,

telefon VARCHAR(30),

datum_uclanjenja DATE NOT NULL,

status VARCHAR(30) NOT NULL

);

7.7. TABLICA zaposlenik

Tablica zaposlenik vodi evidenciju djelatnika knjižnice. Sastoji se od atributa id_zaposlenik, ime, prezime, email, radno_mjesto i datum_zaposlenja. Atribut id_zaposlenik je PRIMARY KEY tipa INT s AUTO_INCREMENT. Atributi ime i prezime su tipa VARCHAR(50) i obavezni. Atribut email je tipa VARCHAR(100), NOT NULL i UNIQUE — kao i kod članova, e-mail jednoznačno identificira zaposlenika. Atribut radno_mjesto je tipa VARCHAR(50) jer naslovi radnih mjesta mogu biti dugački, ovaj atribut je obavezan jer svaki zaposlenik mora imati jasno definiranu poziciju. Atribut datum_zaposlenja je tipa DATE i obavezan — knjižnica uvijek bilježi datum početka radnog odnosa.

CREATE TABLE zaposlenik (

id_zaposlenik INT PRIMARY KEY AUTO_INCREMENT,

```
ime VARCHAR(50) NOT NULL,  
prezime VARCHAR(50) NOT NULL,  
email VARCHAR(100) NOT NULL UNIQUE,  
radno_mjesto VARCHAR(50) NOT NULL,  
datum_zaposlenja DATE NOT NULL  
);
```

7.8. TABLICA knjiga

Tablica knjiga predstavlja bibliografski zapis knjige — to nije fizički primjerak na polici, već apstraktna informacija o naslovu. Sastoji se od atributa id_knjiga, id_izdavac, naslov, isbn, godina_izdanja i broj_stranica. Atribut id_knjiga je PRIMARY KEY tipa INT s AUTO_INCREMENT. Atribut id_izdavac je tipa INT i FOREIGN KEY koji povezuje knjigu s izdavačem; obavezan je (NOT NULL) jer svaka knjiga mora imati izdavača. Atribut naslov je tipa VARCHAR(150) — veća duljina od standardnih 50 jer naslovi knjiga znaju biti dugački ('Harry Potter and the Philosopher Stone'); obavezan je. Atribut isbn je tipa VARCHAR(20), obavezan i UNIQUE. Odabrali smo VARCHAR umjesto BIGINT jer ISBN brojevi mogu počinjati nulom i ponekad sadrže razmake ili crtice. UNIQUE ograničenje je ključno jer je ISBN globalno jedinstven identifikator knjige. Atribut godina_izdanja je tipa INT i opcionalan (može biti NULL za stara djela bez preciznog datuma), no postavljen je CHECK uvjet koji garantira da, ako godina jest unesena, mora biti pozitivan broj — sprječava unos pogrešnih negativnih godina. Slično, atribut broj_stranica je tipa INT, opcionalan, ali s CHECK uvjetom da mora biti pozitivan. Strani ključ na izdavača ima ON UPDATE CASCADE (ako se izdavaču promijeni id, kaskadno se ažurira u knjizi) i ON DELETE RESTRICT (ne može se obrisati izdavač dok god postoje njegove knjige u sustavu — sprječava gubitak referencijalnog integriteta).

CREATE TABLE knjiga (

```

id_knjiga INT PRIMARY KEY AUTO_INCREMENT,

id_izdavac INT NOT NULL,

naslov VARCHAR(150) NOT NULL,

isbn VARCHAR(20) NOT NULL UNIQUE,

godina_izdanja INT,

broj_stranica INT,

FOREIGN KEY (id_izdavac) REFERENCES izdavac(id_izdavac)

ON UPDATE CASCADE

ON DELETE RESTRICT,

CHECK (godina_izdanja IS NULL OR godina_izdanja > 0),

CHECK (broj_stranica IS NULL OR broj_stranica > 0)

);

```

7.9. TABLICA knjiga_autor

Tablica knjiga_autor je vezna (asocijativna) tablica koja realizira many-to-many vezu između knjige i autora — jedna knjiga može imati više autora (koautori), a jedan autor može napisati više knjiga. Sastoji se od tri atributa: id_knjiga, id_autor i redoslijed_autora. Atributi id_knjiga i id_autor su INT i zajedno čine složeni PRIMARY KEY (id_knjiga, id_autor) — ovo sprječava unos dva ista para, tj. ista veza između konkretne knjige i konkretnog autora može postojati samo jednom. Oba atributa su istovremeno i strani ključevi koji referenciraju roditeljske tablice. Atribut redoslijed_autora tipa INT označava redoslijed autorstva (1 = prvi autor, 2 = drugi itd.); ovaj podatak je važan kod citiranja i ispisa knjige. Postavljen je CHECK uvjet da redoslijed mora biti veći od 0. Strani ključ na knjigu ima ON DELETE CASCADE — kad se knjiga obriše, automatski se brišu i sve njezine veze prema autorima jer veza bez knjige nema smisla. Strani ključ na autora ima ON DELETE RESTRICT — ne dopušta se brisanje autora dok god postoji barem jedna knjiga koju je on napisao.

```

CREATE TABLE knjiga_autor (

```

```

id_knjiga INT NOT NULL,

id_autor INT NOT NULL,

redoslijed_autora INT NOT NULL,


PRIMARY KEY (id_knjiga, id_autor),


FOREIGN KEY (id_knjiga) REFERENCES knjiga(id_knjiga)

    ON UPDATE CASCADE

    ON DELETE CASCADE,


FOREIGN KEY (id_autor) REFERENCES autor(id_autor)

    ON UPDATE CASCADE

    ON DELETE RESTRICT,

CHECK (redoslijed_autora > 0)

);

```

7.10. TABLICA knjiga_zanr

Tablica knjiga_zanr je vezna tablica za many-to-many vezu između knjige i žanra — jedna knjiga može pripadati u više žanrova (npr. '1984' je istovremeno roman i znanstvena fantastika), a jedan žanr obuhvaća mnogo knjiga. Sastoji se samo od dva atributa: id_knjiga i id_zanr, oba tipa INT. Zajedno čine složeni PRIMARY KEY (id_knjiga, id_zanr) koji sprječava da se ista veza unese dvaput. Oba atributa su strani ključevi. Atribut redoslijeda nije potreban kao kod autora, jer u kontekstu žanrova nema 'glavnog' i 'sporednog' žanra. Strani ključ prema knjizi ima ON DELETE CASCADE iz istog razloga kao i kod knjiga_autor, a prema žanru ON DELETE RESTRICT kako bi se spriječilo brisanje žanra koji se još koristi.

```

CREATE TABLE knjiga_zanr (

```


id_knjiga INT NOT NULL,

id_zanr INT NOT NULL,

PRIMARY KEY (id_knjiga, id_zanr),

FOREIGN KEY (id_knjiga) REFERENCES knjiga(id_knjiga)

ON UPDATE CASCADE

ON DELETE CASCADE,

FOREIGN KEY (id_zanr) REFERENCES zanr(id_zanr)

ON UPDATE CASCADE

ON DELETE RESTRICT

);

7.11. TABLICA primjerak

Tablica primjerak predstavlja konkretnu fizičku kopiju knjige na polici knjižnice. Važno je razlikovati ovu tablicu od tablice knjiga: knjiga je bibliografski zapis (jedan po naslovu/izdanju), dok primjerak postoji za svaku fizičku kopiju. Tako knjižnica može imati 5 primjeraka iste knjige '1984'. Sastoji se od atributa id_primjerak, id_knjiga, id_lokacija, id_status, inventarni_broj i datum_nabave. Atribut id_primjerak je PRIMARY KEY tipa INT s AUTO_INCREMENT. Atributi id_knjiga, id_lokacija i id_status su tipa INT, sva tri obavezna (NOT NULL) i predstavljaju strane ključeve koji povezuju primjerak s odgovarajućom knjigom, lokacijom u knjižnici i statusom. Atribut inventarni_broj je tipa VARCHAR(30), obavezan i UNIQUE. Odabrali smo VARCHAR jer su inventarni brojevi obično alfanumerički ('INV-0001'); UNIQUE garantira da svaki primjerak ima svoj jedinstveni inventarni broj. Atribut datum_nabave je tipa DATE i obavezan — knjižnica uvijek bilježi datum kad je primjerak nabavljen. Svi strani ključevi imaju ON UPDATE CASCADE i ON DELETE RESTRICT — ne smije se

obrisati ni knjiga, ni lokacija, ni status dok god postoje primjerci koji ih referenciraju, jer bi to ostavilo primjerak bez ispravne reference.

CREATE TABLE primjerak (

id_primjerak INT PRIMARY KEY AUTO_INCREMENT,

id_knjiga INT NOT NULL,

id_lokacija INT NOT NULL,

id_status INT NOT NULL,

inventarni_broj VARCHAR(30) NOT NULL UNIQUE,

datum_nabave DATE NOT NULL,

FOREIGN KEY (id_knjiga) REFERENCES knjiga(id_knjiga)

ON UPDATE CASCADE

ON DELETE RESTRICT,

FOREIGN KEY (id_lokacija) REFERENCES lokacija(id_lokacija)

ON UPDATE CASCADE

ON DELETE RESTRICT,

FOREIGN KEY (id_status) REFERENCES status_primjerka(id_status)

ON UPDATE CASCADE

ON DELETE RESTRICT

);

7.12. TABLICA rezervacija

Tablica rezervacija evidentira zahtjeve članova za rezervaciju određene knjige. Bitno je primijetiti da rezervacija ide na razini KNJIGE (apstraktnog naslova), a ne primjerka — član želi čitati '1984', a knjižnica će mu dodijeliti bilo koji dostupni primjerak. Sastoji se od atributa `id_rezervacija`, `id_clan`, `id_knjiga`, `datum_rezervacije` i `status_rezervacije`. Atribut `id_rezervacija` je PRIMARY KEY tipa INT s AUTO_INCREMENT. Atributi `id_clan` i `id_knjiga` su strani ključevi tipa INT, obavezni (NOT NULL). Atribut `datum_rezervacije` je tipa DATE i obavezan — svaka rezervacija ima datum kad je napravljena. Atribut `status_rezervacije` je tipa VARCHAR(30) i sadrži vrijednosti poput 'Aktivna', 'Istekla', 'Otkazana'. Strani ključevi imaju ON DELETE RESTRICT — ne dopuštamo brisanje člana ili knjige dok god postoji rezervacija koja ih referencira; umjesto brisanja, rezervacija se može označiti kao otkazana.

CREATE TABLE rezervacija (

`id_rezervacija INT PRIMARY KEY AUTO_INCREMENT,`

`id_clan INT NOT NULL,`

`id_knjiga INT NOT NULL,`

`datum_rezervacije DATE NOT NULL,`

`status_rezervacije VARCHAR(30) NOT NULL,`

`FOREIGN KEY (id_clan) REFERENCES clan(id_clan)`

`ON UPDATE CASCADE`

`ON DELETE RESTRICT,`

`FOREIGN KEY (id_knjiga) REFERENCES knjiga(id_knjiga)`

`ON UPDATE CASCADE`

ON DELETE RESTRICT

);

7.13. TABLICA posudba

Tablica posudba predstavlja konkretan čin posuđivanja primjerka knjige od strane člana, koji je realizirao zaposlenik. Sastoji se od atributa id_posudba, id_clan, id_primjerak, id_zaposlenik, datum_posudbe, rok_vracanja, datum_vracanja i status. Atribut id_posudba je PRIMARY KEY tipa INT s AUTO_INCREMENT. Atributi id_clan, id_primjerak i id_zaposlenik su strani ključevi tipa INT, svi obavezni. Atribut datum_posudbe je tipa DATE i obavezan — dan kada je posudba realizirana. Atribut rok_vracanja je tipa DATE i obavezan — označava do kojeg datuma član mora vratiti knjigu. Atribut datum_vracanja je tipa DATE i MOŽE biti NULL — sve dok knjiga nije vraćena, vrijednost je NULL; kada se vrati, upisuje se stvarni datum vraćanja. Atribut status je tipa VARCHAR(30) i obavezan (npr. 'Aktivno', 'Vraceno'). Tablica ima dva CHECK ograničenja koja sprječavaju logičke pogreške: prvi garantira da rok vraćanja ne može biti prije datuma posudbe ($\text{rok_vracanja} \geq \text{datum_posudbe}$), a drugi da datum vraćanja, kada se unese, ne može biti prije datuma posudbe. Sva tri strana ključa imaju ON DELETE RESTRICT jer brisanje člana, primjerka ili zaposlenika ne smije ostaviti posudbu bez konteksta — povijest posudbi treba ostati očuvana radi revizije.

CREATE TABLE posudba (

id_posudba INT PRIMARY KEY AUTO_INCREMENT,

id_clan INT NOT NULL,

id_primjerak INT NOT NULL,

id_zaposlenik INT NOT NULL,

datum_posudbe DATE NOT NULL,

rok_vracanja DATE NOT NULL,

datum_vracanja DATE,

status VARCHAR(30) NOT NULL,

```

FOREIGN KEY (id_clan) REFERENCES clan(id_clan)

ON UPDATE CASCADE ON DELETE RESTRICT,

FOREIGN KEY (id_primjerak) REFERENCES primjerak(id_primjerak)

ON UPDATE CASCADE ON DELETE RESTRICT,

FOREIGN KEY (id_zaposlenik) REFERENCES zaposlenik(id_zaposlenik)

ON UPDATE CASCADE ON DELETE RESTRICT,

CHECK (rok_vracanja >= datum_posudbe),

CHECK (datum_vracanja IS NULL OR datum_vracanja >= datum_posudbe)

);

```

7.14. TABLICA razlog_kazne

Tablica razlog_kazne je šifarnik za razloga kazne. Sastoji se od atributa id_razlog, naziv_razloga i osnovna_cijena. Atribut id_razlog je PRIMARY KEY tipa INT s AUTO_INCREMENT. Atribut naziv_razloga je tipa VARCHAR(30), obavezan i UNIQUE — tablica sadrži tri vrijednosti ('Kasnjenje', 'Ostecenje', 'Gubitak') i UNIQUE garantira da se nijedan razlog ne može upisati dvaput. Atribut osnovna_cijena je tipa NUMERIC(8,2). Atribut je obavezan, a CHECK ograničenje (osnovna_cijena >= 0) sprječava unos negativnih iznosa. Za kašnjenje ovaj iznos predstavlja cijenu po danu (0.50 EUR), a za oštećenje i gubitak fiksne iznose (5.00 odnosno 20.00 EUR).

```

CREATE TABLE razlog_kazne (

    id_razlog INT PRIMARY KEY AUTO_INCREMENT,

    naziv_razloga VARCHAR(30) NOT NULL UNIQUE,

    osnovna_cijena NUMERIC(8,2) NOT NULL,

```

CHECK (osnovna_cijena >= 0)

);

7.15. TABLICA kazna

Tablica kazna evidentira sve kazne izrečene članovima. Sastoji se od atributa id_kazna, id_posudba, id_razlog, iznos, datum i placeno. Atribut id_kazna je PRIMARY KEY tipa INT s AUTO_INCREMENT. Atribut id_posudba je strani ključ tipa INT koji povezuje kaznu s posudbom iz koje je proizašla; obavezan je. Atribut id_razlog je strani ključ tipa INT koji upućuje na šifarnik razloga; također obavezan. Postavljeno je vrlo bitno UNIQUE ograničenje na kombinaciju (id_posudba, id_razlog) — jedna posudba može imati VIŠE kazni, ali ne dvije kazne ISTOG razloga. Atribut iznos je tipa NUMERIC(8,2) — isto obrazloženje kao za osnovnu_cijenu u tablici razloga. CHECK ograničenje (iznos >= 0) sprječava unos negativnih iznosa. Atribut datum je tipa DATE i obavezan — označava kad je kazna evidentirana. Atribut placeno je tipa BOOLEAN s DEFAULT FALSE — sve dok član ne plati kaznu, vrijednost je FALSE; kad plati, postavlja se na TRUE. Strani ključ na posudbu ima ON DELETE CASCADE — ako se posudba obriše, automatski se brišu i njezine kazne (jer kazna bez posudbe nema smisla). Strani ključ na razlog ima ON DELETE RESTRICT — ne smije se obrisati razlog dok god postoje kazne koje ga koriste.

CREATE TABLE kazna (

id_kazna INT PRIMARY KEY AUTO_INCREMENT,

id_posudba INT NOT NULL,

id_razlog INT NOT NULL,

iznos NUMERIC(8,2) NOT NULL,

datum DATE NOT NULL,

placeno BOOLEAN NOT NULL DEFAULT FALSE,

UNIQUE (id_posudba, id_razlog),

```
FOREIGN KEY (id_posudba) REFERENCES posudba(id_posudba)
```

```
ON UPDATE CASCADE ON DELETE CASCADE,
```

```
FOREIGN KEY (id_razlog) REFERENCES razlog_kazne(id_razlog)
```

```
ON UPDATE CASCADE ON DELETE RESTRICT,
```

```
CHECK (iznos >= 0)
```

```
);
```

8. Pogledi i upiti

8.1 Lorena Pavličić

Modul Bibliografski katalog obuhvaća relacije AUTOR, IZDAVAC, KNJIGA i KNJIGA_AUTOR. Služi za organizaciju i pregled osnovnih bibliografskih podataka, a omogućuje evidenciju knjiga, autora i izdavača te povezivanje knjiga s jednim ili više autora.

8.1.1 Pogled 1: v_bibliografski_katalog

Pogled bibliografskog kataloga knjiga

U knjižnici se vodi evidencija osnovnih bibliografskih podataka o svakoj knjizi. Budući da jedna knjiga može imati jednog ili više autora, a svaki autor može biti povezan s više knjiga, potrebno je omogućiti pregled knjiga zajedno s pripadajućim autorima i izdavačima. Kako bi se omogućio takav pregled, sustav mora prikazati osnovne podatke o knjizi, izdavaču i autoru, uključujući naslov knjige, ISBN, godinu izdanja, broj stranica, naziv izdavača, državu izdavača, ime i prezime autora te redoslijed autora kod knjiga koje imaju više autora.

```
CREATE OR REPLACE VIEW v_bibliografski_katalog AS
SELECT
    k.id_knjiga,
    k.naslov,
    k.isbn,
    k.godina_izdanja,
    k.broj_stranica,
    i.naziv AS izdavac,
    i.drzava AS drzava_izdavaca,
    CONCAT(a.ime, ' ', a.prezime) AS autor,
    a.drzava AS drzava_autora,
```

```

    ka.redoslijed_autora
FROM knjiga k
JOIN izdavic i ON k.id_izdavic = i.id_izdavic
JOIN knjiga_autor ka ON k.id_knjiga = ka.id_knjiga
JOIN autor a ON ka.id_autor = a.id_autor;

```

TRAZENO RJEŠENJE: id_knjiga, naslov, isbn, godina_izdanja, broj_stranica, izdavic, drzava_izdavaca, autor, drzava_autora, redoslijed_autora

Opis pogleda:

- **FROM knjiga AS k** - koristim tablicu knjiga kao glavnu tablicu jer ona sadrži osnovne bibliografske podatke o svakoj knjizi u knjižničnom katalogu, kao što su naslov, ISBN, godina izdanja i broj stranica.
- **INNER JOIN izdavic AS i ON k.id_izdavic = i.id_izdavic** - tablicu izdavic povezujem s tablicom knjiga preko atributa id_izdavic kako bih za svaku knjigu mogao prikazati naziv izdavača i državu izdavača.
- **INNER JOIN knjiga_autor AS ka ON k.id_knjiga = ka.id_knjiga** - tablicu knjiga_autor povezujem s tablicom knjiga preko atributa id_knjiga jer ta povezna relacija omogućuje prikaz autora koji su povezani s pojedinom knjigom.
- **INNER JOIN autor AS a ON ka.id_autor = a.id_autor** - tablicu autor povezujem s tablicom knjiga_autor preko atributa id_autor kako bih mogao prikazati ime, prezime i državu autora svake knjige.
- **SELECT** - u SELECT dijelu biram podatke koje želim prikazati u rezultatu pregleda, a to su ID knjige, naslov, ISBN, godina izdanja, broj stranica, naziv izdavača, država izdavača, ime i prezime autora, država autora te redoslijed autora.
- **i.naziv AS izdavic** - atribut naziv iz tablice izdavic preimenovao sam u izdavic kako bi naziv stupca bio jasniji u rezultatu pregleda.
- **i.drzava AS drzava_izdavaca** - atribut drzava iz tablice izdavic preimenovao sam u drzava_izdavaca kako bi se razlikovao od države autora.
- **CONCAT(a.ime, ' ', a.prezime) AS autor** - pomoću funkcije CONCAT spajam ime i prezime autora u jedan stupac pod nazivom autor, čime je rezultat pregleda čitljiviji.
- **a.drzava AS drzava_autora** - atribut drzava iz tablice autor preimenovao sam u drzava_autora kako bi bilo jasno da se taj podatak odnosi na autora, a ne na izdavača.
- **ka.redoslijed_autora** - prikazujem atribut redoslijed_autora iz tablice knjiga_autor jer on označava redoslijed autora kod knjiga koje imaju više autora.
- **DROP VIEW IF EXISTS v_bibliografski_katalog** - pomoću ove naredbe brišem pogled ako već postoji, kako bih ga mogao ponovno kreirati bez greške prilikom ponovnog pokretanja SQL skripte.

- **CREATE VIEW v_bibliografski_katalog** - pomoću naredbe CREATE VIEW kreiram pogled pod nazivom v_bibliografski_katalog, koji služi za pregled osnovnih bibliografskih podataka o knjigama, autorima i izdavačima u knjižničnom sustavu.

8.1.2 Pogled 2: v_broj_knjiga_po_autoru

Analiza broja knjiga po autoru

U knjižnici se vodi evidencija autora i knjiga koje su povezane s njima. Budući da jedan autor može napisati više knjiga, potrebno je analizirati koliko je knjiga u katalogu povezano sa svakim autorom. Kako bi se omogućila takva analiza, sustav prvo mora omogućiti pregled svih autora zajedno s brojem knjiga koje su evidentirane pod njihovim imenom. Time se može utvrditi koji autori imaju najveći broj knjiga u knjižničnom katalogu, ali i postoje li autori koji trenutno nemaju povezanu nijednu knjigu.

TRAŽENO RJEŠENJE: id_autor, autor, drzava, broj_knjiga

```
CREATE OR REPLACE VIEW v_broj_knjiga_po_autoru AS
SELECT
    a.id_autor,
    CONCAT(a.ime, ' ', a.prezime) AS autor,
    a.drzava,
    COUNT(ka.id_knjiga) AS broj_knjiga
FROM autor a
LEFT JOIN knjiga_autor ka ON a.id_autor = ka.id_autor
GROUP BY a.id_autor, a.ime, a.prezime, a.drzava;
```

Opis pogleda:

- **FROM autor AS a** - koristim tablicu autor kao glavnu tablicu jer ona sadrži osnovne podatke o svakom autoru koji je evidentiran u knjižničnom sustavu.
- **LEFT JOIN knjiga_autor AS ka ON a.id_autor = ka.id_autor** - tablicu knjiga_autor povezujem s tablicom autor preko atributa id_autor kako bih mogao prebrojati koliko je knjiga povezano sa svakim autorom.
- **LEFT JOIN** - koristim LEFT JOIN zato što želim prikazati sve autore iz tablice autor, čak i one koji trenutno nemaju povezanu nijednu knjigu u tablici knjiga_autor.

- **SELECT** - u SELECT dijelu biram podatke koje želim prikazati u rezultatu pregleda, a to su ID autora, puno ime autora, država autora i broj knjiga povezanih s autorom.
- **CONCAT(a.ime, ' ', a.prezime) AS autor** - pomoću funkcije CONCAT spajam ime i prezime autora u jedan stupac pod nazivom autor, čime je rezultat pregleda jednostavniji i čitljiviji.
- **a.drzava** - prikazujem državu autora kako bi se uz broj knjiga mogao vidjeti i dodatni podatak o autoru.
- **COUNT(ka.id_knjiga) AS broj_knjiga** - pomoću agregatne funkcije COUNT brojim koliko je knjiga povezano sa svakim autorom preko tablice knjiga_autor. Rezultat preimenujem u broj_knjiga radi jasnijeg prikaza.
- **GROUP BY a.id_autor, a.ime, a.prezime, a.drzava** - koristim GROUP BY kako bih rezultate grupirao po svakom autoru i za svakog autora izračunao ukupan broj povezanih knjiga.
- **DROP VIEW IF EXISTS v_broj_knjiga_po_autoru** - pomoću ove naredbe brišem pogled ako već postoji, kako bih ga mogao ponovno kreirati bez greške prilikom ponovnog pokretanja SQL skripte.
- **CREATE VIEW v_broj_knjiga_po_autoru** - pomoću naredbe CREATE VIEW kreiram pogled pod nazivom v_broj_knjiga_po_autoru, koji služi za analizu broja knjiga po autoru u bibliografskom katalogu.

8.1.3 Pogled 3: v_broj_autora_po_knjizi

Analiza knjiga s više autora

U knjižnici se za svaku knjigu vodi evidencija autora koji su sudjelovali u njezinu pisanju. Budući da jedna knjiga može imati jednog ili više autora, potrebno je analizirati koliko autora ima svaka knjiga u bibliografskom katalogu. Kako bi se omogućila takva analiza, sustav prvo mora omogućiti pregled svih knjiga zajedno s nazivom izdavača i brojem autora koji su povezani s pojedinom knjigom. Na taj način moguće je prepoznati knjige koje imaju više autora te jasnije prikazati vezu više-na-više između relacija AUTOR i KNJIGA.

TRAŽENO RJEŠENJE: id_knjiga, naslov, godina_izdanja, izdavac, broj_autora

```
CREATE OR REPLACE VIEW v_broj_autora_po_knjizi AS
SELECT
    k.id_knjiga,
    k.naslov,
    k.godina_izdanja,
    i.naziv AS izdavac,
    COUNT(ka.id_autor) AS broj_autora
FROM knjiga k
JOIN izdavac i ON k.id_izdavac = i.id_izdavac
LEFT JOIN knjiga_autor ka ON k.id_knjiga = ka.id_knjiga
GROUP BY k.id_knjiga, k.naslov, k.godina_izdanja, i.naziv;
```

Opis pogleda:

- **FROM knjiga AS k** - koristim tablicu knjiga kao glavnu tablicu jer ona sadrži osnovne bibliografske podatke o svakoj knjizi u knjižničnom katalogu.
- **INNER JOIN izdavac AS i ON k.id_izdavac = i.id_izdavac** - tablicu izdavac povezujem s tablicom knjiga preko atributa id_izdavac kako bih za svaku knjigu mogao prikazati naziv izdavača.
- **LEFT JOIN knjiga_autor AS ka ON k.id_knjiga = ka.id_knjiga** - tablicu knjiga_autor povezujem s tablicom knjiga preko atributa id_knjiga kako bih mogao prebrojati koliko je autora povezano sa svakom knjigom.
- **LEFT JOIN** - koristim LEFT JOIN zato što želim prikazati sve knjige iz tablice knjiga, čak i ako neka knjiga trenutno nema povezanog autora u tablici knjiga_autor.
- **SELECT** - u SELECT dijelu biram podatke koje želim prikazati u rezultatu pregleda, a to su ID knjige, naslov knjige, godina izdanja, naziv izdavača i broj autora povezanih s knjigom.
- **i.naziv AS izdavac** - atribut naziv iz tablice izdavac preimenovao sam u izdavac kako bi naziv stupca bio jasniji i pregledniji u rezultatu pregleda.
- **COUNT(ka.id_autor) AS broj_autora** - pomoću agregatne funkcije COUNT brojim koliko je autora povezano sa svakom knjigom preko tablice knjiga_autor. Rezultat preimenujem u broj_autora radi jasnijeg prikaza.
- **GROUP BY k.id_knjiga, k.naslov, k.godina_izdanja, i.naziv** - koristim GROUP BY kako bih rezultate grupirao po svakoj knjizi i za svaku knjigu izračunao ukupan broj povezanih autora.
- **DROP VIEW IF EXISTS v_broj_autora_po_knjizi** - pomoću ove naredbe brišem pogled ako već postoji, kako bih ga mogao ponovno kreirati bez greške prilikom ponovnog pokretanja SQL skripte.
- **CREATE VIEW v_broj_autora_po_knjizi** - pomoću naredbe CREATE VIEW kreiram pogled pod nazivom v_broj_autora_po_knjizi, koji služi za analizu broja autora po knjizi i prikaz knjiga koje imaju jednog ili više autora.

8.1.4 Upit 1: Prikazuje sve knjige određenog autora

U knjižnici se često javlja potreba za pretraživanjem knjiga prema autoru. Budući da jedan autor može napisati više knjiga, a jedna knjiga može imati više autora, potrebno je omogućiti pregled svih knjiga koje su povezane s određenim autorom u bibliografskom katalogu. Kako bi se omogućila takva pretraga, sustav mora povezati podatke o autorima, knjigama i izdavačima. Na taj način knjižničar ili korisnik može unosom prezimena autora dobiti popis svih njegovih knjiga, zajedno s osnovnim bibliografskim podacima kao što su naslov, ISBN, godina izdanja i naziv izdavača.

```

SELECT
    CONCAT(a.ime, ' ', a.prezime) AS autor,
    k.naslov,
    k.isbn,
    k.godina_izdanja,
    i.naziv AS izdavac
FROM autor a
JOIN knjiga_autor ka ON a.id_autor = ka.id_autor
JOIN knjiga k ON ka.id_knjiga = k.id_knjiga
JOIN izdavac i ON k.id_izdavac = i.id_izdavac
WHERE a.prezime = 'Orwell'
ORDER BY k.godina_izdanja DESC;

```

TRAŽENO RJEŠENJE: autor, naslov, isbn, godina_izdanja, izdavac

Opis upita:

- **FROM autor AS a** - koristim tablicu autor kao početnu tablicu jer se pretraga temelji na autoru, odnosno na njegovom prezimenu.
- **INNER JOIN knjiga_autor AS ka ON a.id_autor = ka.id_autor** - tablicu knjiga_autor povezujem s tablicom autor preko atributa id_autor kako bih pronašao sve knjige koje su povezane s određenim autorom.
- **INNER JOIN knjiga AS k ON ka.id_knjiga = k.id_knjiga** - tablicu knjiga povezujem s tablicom knjiga_autor preko atributa id_knjiga kako bih za svaku pronađenu vezu mogao prikazati bibliografske podatke o knjizi.
- **INNER JOIN izdavac AS i ON k.id_izdavac = i.id_izdavac** - tablicu izdavac povezujem s tablicom knjiga preko atributa id_izdavac kako bih mogao prikazati naziv izdavača za svaku knjigu.
- **SELECT** - u SELECT dijelu biram podatke koje želim prikazati u rezultatu upita, a to su puno ime autora, naslov knjige, ISBN, godina izdanja i naziv izdavača.
- **CONCAT(a.ime, ' ', a.prezime) AS autor** - pomoću funkcije CONCAT spajam ime i prezime autora u jedan stupac pod nazivom autor, čime rezultat upita postaje pregledniji.
- **WHERE a.prezime = 'Orwell'** - koristim WHERE uvjet kako bih izdvojio samo knjige autora čije je prezime Orwell. Ovaj uvjet se može promijeniti ovisno o autoru kojeg korisnik želi pretražiti.
- **ORDER BY k.godina_izdanja DESC** - rezultate sortiram silazno prema godini izdanja kako bi se najnovije knjige odabranog autora prikazale prve.
- **Svrha upita** - upit omogućuje pretragu bibliografskog kataloga prema autoru i prikazuje sve knjige povezane s tim autorom, zajedno s osnovnim bibliografskim podacima i izdavačem.

8.1.5 Upit 2: Prikazuje autore koji imaju dvije ili više knjiga u katalogu.

Korištenjem klauzule HAVING pronalazimo autore koji u katalogu imaju najmanje dvije knjige. U knjižnici se često javlja potreba za analizom autora prema broju knjiga koje su povezane s njima u bibliografskom katalogu. Budući da jedan autor može napisati više knjiga, potrebno je omogućiti pregled autora koji imaju veći broj knjiga u knjižničnom fondu. Kako bi se omogućila takva analiza, sustav mora povezati podatke o autorima i knjigama preko povezne relacije KNJIGA_AUTOR. Na taj način knjižničar može vidjeti koji su autori najzastupljeniji u katalogu, iz koje države dolaze i koliko je knjiga povezano s pojedinim autorom. Upit dodatno izdvaja samo one autore koji imaju dvije ili više knjiga, čime se dobiva pregled autora s većim doprinosom knjižničnom fondu.

TRAŽENO RJEŠENJE: autor, drzava, broj_knjiga

```
SELECT
    CONCAT(a.ime, ' ', a.prezime) AS autor,
    a.drzava,
    COUNT(ka.id_knjiga) AS broj_knjiga
FROM autor a
JOIN knjiga_autor ka ON a.id_autor = ka.id_autor
GROUP BY a.id_autor, a.ime, a.prezime, a.drzava
HAVING COUNT(ka.id_knjiga) >= 2
ORDER BY broj_knjiga DESC, autor;
```

Opis upita:

- **FROM autor AS a** - koristim tablicu autor kao početnu tablicu jer se upit temelji na analizi autora i broju knjiga koje su povezane s njima.
- **INNER JOIN knjiga_autor AS ka ON a.id_autor = ka.id_autor** - tablicu knjiga_autor povezujem s tablicom autor preko atributa id_autor kako bih mogao prebrojati koliko je knjiga povezano sa svakim autorom.
- **SELECT** - u SELECT dijelu biram podatke koje želim prikazati u rezultatu upita, a to su puno ime autora, država autora i broj knjiga povezanih s autorom.
- **CONCAT(a.ime, ' ', a.prezime) AS autor** - pomoću funkcije CONCAT spajam ime i prezime autora u jedan stupac pod nazivom autor, čime rezultat upita postaje čitljiviji.
- **a.drzava** - prikazujem državu autora kako bi se uz broj knjiga dobio dodatni podatak o autoru.
- **COUNT(ka.id_knjiga) AS broj_knjiga** - pomoću agregatne funkcije COUNT brojim koliko je knjiga povezano sa svakim autorom preko tablice knjiga_autor. Rezultat preimenujem u broj_knjiga radi jasnijeg prikaza.

- **GROUP BY a.id_autor, a.ime, a.prezime, a.drzava** - koristim GROUP BY kako bih grupirala rezultate po svakom autoru i za svakog autora izračunala ukupan broj knjiga.
- **HAVING COUNT(ka.id_knjiga) >= 2** - koristim HAVING uvjet kako bih izdvojila samo one autore koji imaju dvije ili više knjiga u katalogu. HAVING se koristi zato što se uvjet odnosi na agregatnu vrijednost dobivenu funkcijom COUNT.
- **ORDER BY broj_knjiga DESC, autor ASC** - rezultate sortiram silazno prema broju knjiga kako bi autori s najviše knjiga bili prikazani prvi, a zatim uzlazno prema imenu autora radi preglednijeg prikaza.

8.1.6 Upit 3: Prikazuje knjige koje imaju više od jednog autora.

U knjižnici se često javlja potreba za analizom knjiga koje imaju više autora. Budući da jedna knjiga može imati jednog ili više autora, potrebno je omogućiti pregled knjiga kod kojih je u bibliografskom katalogu evidentirano više od jednog autora. Kako bi se omogućila takva analiza, sustav mora povezati podatke o knjigama, izdavačima i autorima preko povezne relacije KNJIGA_AUTOR. Na taj način knjižničar može vidjeti koje knjige imaju više autora, iz koje su godine izdanja, kojem izdavaču pripadaju i koliko je autora povezano sa svakom knjigom. Upit koristi grupiranje knjiga i izdavača samo one knjige kod kojih je broj autora veći od jedan.

TRAŽENO RJEŠENJE: naslov, godina_izdanja, izdavač, broj_autora

```
SELECT
    k.naslov,
    k.godina_izdanja,
    i.naziv AS izdavač,
    COUNT(ka.id_autor) AS broj_autora
FROM knjiga k
JOIN izdavač i ON k.id_izdavač = i.id_izdavač
JOIN knjiga_autor ka ON k.id_knjiga = ka.id_knjiga
GROUP BY k.id_knjiga, k.naslov, k.godina_izdanja, i.naziv
HAVING COUNT(ka.id_autor) > 1
ORDER BY broj_autora DESC, k.naslov;
```

Opis upita:

- **FROM knjiga AS k** - koristim tablicu knjiga kao početnu tablicu jer se upit temelji na analizi knjiga i broju autora koji su povezani s pojedinom knjigom.

- **INNER JOIN izdavac AS i ON k.id_izdavac = i.id_izdavac** - tablicu izdavac povezujem s tablicom knjiga preko atributa id_izdavac kako bih za svaku knjigu mogao prikazati naziv izdavača.
- **INNER JOIN knjiga_autor AS ka ON k.id_knjiga = ka.id_knjiga** - tablicu knjiga_autor povezujem s tablicom knjiga preko atributa id_knjiga kako bih mogao prebrojati koliko je autora povezano sa svakom knjigom.
- **SELECT** - u SELECT dijelu biram podatke koje želim prikazati u rezultatu upita, a to su naslov knjige, godina izdanja, naziv izdavača i broj autora povezanih s knjigom.
- **i.naziv AS izdavac** - atribut naziv iz tablice izdavac preimenovao sam u izdavac kako bi naziv stupca bio jasniji u rezultatu upita.
- **COUNT(ka.id_autor) AS broj_autora** - pomoću agregatne funkcije COUNT brojim koliko je autora povezano sa svakom knjigom preko tablice knjiga_autor. Rezultat preimenujem u broj_autora radi preglednijeg prikaza.
- **GROUP BY k.id_knjiga, k.naslov, k.godina_izdanja, i.naziv** - koristim GROUP BY kako bih rezultate grupirao po svakoj knjizi i za svaku knjigu izračunao ukupan broj povezanih autora.
- **HAVING COUNT(ka.id_autor) > 1** - koristim HAVING uvjet kako bih izdvojio samo one knjige koje imaju više od jednog autora. HAVING se koristi zato što se uvjet odnosi na agregatnu vrijednost dobivenu funkcijom COUNT.
- **ORDER BY broj_autora DESC, k.naslov ASC** - rezultate sortiram silazno prema broju autora kako bi knjige s najviše autora bile prikazane prve, a zatim uzlazno prema naslovu knjige radi preglednijeg prikaza rezultata.

8.2 Lucija Baljak

Modul obuhvaća relacije: POSUDBA, ZAPOSLENIK, KAZNA, RAZLOG KAZNE, CLAN, KNJIGA, PRIMJERAK. Služi za pregled kazni, statistike kazni i statistike zaposlenika.

8.2.1 Pogled 1: ocekivane_kazne

Kada član dođe u knjižnicu po novu posudbu, knjižničar treba brzo provjeriti ima li aktivnih kašnjenja i koliki iznos kazne treba naplatiti prije nego što odobri novu posudbu. View spaja podatke o posudbi, članu, knjizi i cijeni kazne u jedan pregled, s automatski izračunatim iznosom na temelju broja dana kašnjenja i osnovne cijene iz tablice razlog_kazne.

```
CREATE OR REPLACE VIEW ocekivane_kazne AS
```

```

SELECT

    c.id_clan,

    CONCAT(c.ime, ' ', c.prezime) AS clan,

    c.email,

    pk.inventarni_broj,

    k.naslov,

    p.rok_vracanja,

    DATEDIFF(CURDATE(), p.rok_vracanja) AS dana_kasnjenja,

    rk.osnovna_cijena,

    GREATEST(DATEDIFF(CURDATE(), p.rok_vracanja) * rk.osnovna_cijena, 0) AS
izracunata_kazna
FROM posudba AS p

INNER JOIN primjerak AS pk ON pk.id_primjerak = p.id_primjerak

INNER JOIN knjiga AS k ON pk.id_knjiga = k.id_knjiga

INNER JOIN clan AS c ON c.id_clan = p.id_clan

CROSS JOIN razlog_kazne AS rk

WHERE p.status = 'Aktivno'

    AND DATEDIFF(CURDATE(), p.rok_vracanja) > 0

    AND rk.naziv_razloga = 'Kasnjenje'

```

Opis pogleda:

- U FROM dijelu navedena je tablica posudba pod aliasom p kao polazna točka upita jer sadrži sve ključne podatke o posudbama.
- Tablica posudba povezana je pomoću INNER JOIN s tablicama primjerak (alias pk), knjiga (alias k) i clan (alias c) po pripadajućim ID atributima. INNER JOIN korišten je jer su nam potrebni samo redovi koji imaju podudaranje u svim spojenim tablicama — posudba bez primjerka, knjige ili člana nema smisla prikazivati.
- Tablica razlog_kazne spojena je pomoću CROSS JOIN jer ne postoji direktni ključ između posudbe i razloga kazne — koristi se isključivo kao lookup za dohvat osnovne cijene kašnjenja. WHERE uvjet odmah ograničava rezultat samo na redak s nazivom 'Kasnjenje' kako ne bismo dobili višestruke retke po

posudbi.

- U WHERE dijelu postavljena su tri uvjeta spojena AND operatorom: p.status = 'Aktivno' zadržava samo nevraćene posudbe, DATEDIFF(CURDATE(), p.rok_vracanja) > 0 zadržava samo one koje kasne, a rk.naziv_razloga = 'Kasnjenje' ograničava CROSS JOIN na jedan redak.
- U SELECT dijelu navedeni su svi atributi potrebni knjižničaru: podaci o članu (id_clan za filtriranje, ime i prezime spojeni CONCAT-om, email), podaci o knjizi (naslov, inventarni_broj), rok vraćanja i dana kašnjenja. Izračunata kazna dobiva se množenjem dana kašnjenja i osnovne cijene, a GREATEST osigurava da rezultat ne bude negativan.
- Sa naredbom CREATE OR REPLACE VIEW kreiran je pogled nazvan ocekivane_kazne kako bi knjižničar imao uvijek ažuran pregled kašnjenja s izračunatim iznosima.

8.2.2 Select 1: Pregled kašnjenja po članu

Kada član dođe u knjižnicu, knjižničar pretražuje njegova aktivna kašnjenja unosom id_clan kako bi vidio sve nevrađene knjige, koliko dana kasni i koliki iznos treba naplatiti prije odobravanja nove posudbe. Alternativno, pretraga je moguća i po inventarnom broju kopije.

```
SELECT
    ok.clan,
    ok.inventarni_broj,
    ok.naslov,
    ok.rok_vracanja,
    ok.dana_kasnjenja,
    ok.osnovna_cijena,
    ok.izracunata_kazna
FROM ocekivane_kazne AS ok
WHERE ok.id_clan = 1
```

```
ORDER BY ok.dana_kasnjenja DESC;
```

Opis upita:

- U FROM dijelu podaci se čitaju iz pogleda ocekivane_kazne pod aliasom ok.
- U WHERE dijelu filtrira se po id_clan željenog člana kako bi se prikazala samo njegova kašnjenja.
- U ORDER BY dijelu rezultati su sortirani silazno po danima kašnjenja kako bi knjižničar odmah vidio najdulje kašnjenje i znao što prioritetno naplatiti.
- U SELECT dijelu navedeni su svi atributi relevantni za knjižničara: kontakt podaci člana, podaci o knjizi, rok vraćanja, broj dana kašnjenja i izračunati iznos kazne.

8.2.3 Pogled 2: zaposlenici_statistika

Voditelj knjižnice treba pratiti učinkovitost zaposlenika — ne samo koliko knjiga izdaju mjesečno, nego i kakav je ishod tih posudbi. Zaposlenik koji izdaje mnogo knjiga ali velik dio završi kašnjenjem, oštećenjem ili gubitkom. Ovaj view daje mjesečnu statistiku svakog zaposlenika koja se može koristiti za godišnje evaluacije, dodjelu bonusa ili identifikaciju problema u radu.

```
CREATE OR REPLACE VIEW zaposlenici_statistika AS
SELECT
    z.id_zaposlenik,
    CONCAT(z.ime, ' ', z.prezime) AS zaposlenik,
    z.radno_mjesto,
    z.datum_zaposlenja,
    DATE_FORMAT(p.datum_posudbe, '%Y') AS godina,
    DATE_FORMAT(p.datum_posudbe, '%Y-%m') AS mjesec,
```

```

COUNT(DISTINCT p.id_posudba) AS izdano_knjiga,

SUM(CASE WHEN p.datum_vracanja IS NOT NULL AND ka.id_kazna IS NULL THEN 1
ELSE 0 END) AS vraceno_bez_kazne,

COUNT(DISTINCT CASE WHEN ka.id_kazna IS NOT NULL THEN p.id_posudba END) AS
posudbi_s_kaznom,

SUM(CASE WHEN rk.naziv_razloga = 'Kasnjenje' THEN 1 ELSE 0 END) AS
kazni_kasnjenje,

SUM(CASE WHEN rk.naziv_razloga = 'Ostecenje' THEN 1 ELSE 0 END) AS
kazni_ostecenje,

SUM(CASE WHEN rk.naziv_razloga = 'Gubitak' THEN 1 ELSE 0 END) AS
kazni_gubitak,

COALESCE(SUM(ka.iznos), 0) AS obracunato_kazni,

COALESCE(SUM(CASE WHEN ka.placeno = TRUE THEN ka.iznos ELSE 0 END), 0) AS
naplaceno_kazni
FROM zaposlenik AS z
INNER JOIN posudba AS p ON z.id_zaposlenik = p.id_zaposlenik
LEFT JOIN kazna AS ka ON p.id_posudba = ka.id_posudba
LEFT JOIN razlog_kazne AS rk ON rk.id_razlog = ka.id_razlog
GROUP BY z.id_zaposlenik, godina, mjesec;

```

Opis pogleda:

- Unutar glavnog upita, u FROM dijelu, navela sam tablicu zaposlenik pod aliasom z, koja mi služi kao polazna točka.
- Sa INNER JOIN naredbom povezala sam tablicu posudba pod aliasom p jer trebam vidjeti koje je posudbe koji zaposlenik obradio.
- Nakon toga sam dodala LEFT JOIN te povezala i tablicu kazna pod aliasom ka jer trebam vidjeti koje posudbe su završile s kaznom. Koristila sam LEFT JOIN zato što većina posudbi neće imati kaznu a želim zadržati i posudbe koje nemaju kaznu. Uvjet spajanja je po atributu id_posudba iz tablica posudba i kazna.
- Drugi LEFT JOIN sam koristila da spojim i tablicu razlog_kazne pod aliasom rk jer trebam naziv razloga kazne — da mogu razdvojiti kazne po tipu (Kasnjenje, Ostecenje, Gubitak).

- U GROUP BY dijelu grupiram rezultate po tri stvari: z.id_zaposlenik, godina i mjesec. To znači da za svakog zaposlenika dobivam jedan redak po mjesecu.
- U SELECT dijelu navela sam sve potrebne attribute: z.id_zaposlenik i CONCAT(z.ime, ' ', z.prezime) AS zaposlenik prikazuju identifikator i ime zaposlenika, z.radno_mjesto pokazuje poziciju, a z.datum_zaposlenja pokazuje od kada je zaposlenik u knjižnici.
- DATE_FORMAT funkcija koristi se za izvlačenje godine i mjeseca iz datuma posudbe — DATE_FORMAT(p.datum_posudbe, '%Y') izvlači samo godinu (npr. '2025'), a DATE_FORMAT(p.datum_posudbe, '%Y-%m') izvlači godinu i mjesec (npr. '2025-10'). Godina mi služi za filtriranje kasnije u SELECT-u za ovaj pogled, a mjesec za detaljnije grupiranje statistike.
- COUNT(DISTINCT p.id_posudba) AS izdano_knjiga broji koliko različitih posudbi je zaposlenik obradio taj mjesec. DISTINCT je bitan jer jedna posudba može imati više kazni — bez DISTINCT bi se ista posudba brojala više puta, jednom za svaku kaznu, što bi dalo krivi rezultat.
- SUM(CASE WHEN p.datum_vracanja IS NOT NULL AND ka.id_kazna IS NULL THEN 1 ELSE 0 END) AS vraceno_bez_kazne broji posudbe koje su uredno vraćene bez ikakve kazne.
- COUNT(DISTINCT CASE WHEN ka.id_kazna IS NOT NULL THEN p.id_posudba END) AS posudbi_s_kaznom broji koliko različitih posudbi je završilo s barem jednom kaznom.
- SUM(CASE WHEN rk.naziv_razloga = 'Kasnjenje' THEN 1 ELSE 0 END) AS kazni_kasnjenje razdvaja kazne po tipu. Ista logika se primjenjuje za kazni_ostecenje i kazni_gubitak.
- COALESCE(SUM(ka.iznos), 0) AS obracunato_kazni je ukupan iznos svih kazni obračunatih na posudbama tog zaposlenika, a COALESCE(SUM(CASE WHEN ka.placeno = TRUE THEN ka.iznos ELSE 0 END) AS naplaceno_kazni je koliko je od toga stvarno naplaćeno. Razlika između ta dva broja pokazuje koliko novca knjižnica još čeka. COALESCE pretvara NULL u 0 za zaposlenike čije posudbe nemaju kazne.
- Sa naredbom CREATE OR REPLACE VIEW kreiran je pogled nazvan zaposlenici_statistika kako bi se pratila mjesečna učinkovitost svakog zaposlenika.

8.2.4 Select 2: Učinkovitost zaposlenika — godišnji izvještaj za 2025.

Na kraju godine voditelj knjižnice želi rangirati zaposlenike po učinkovitosti. Ne gleda se samo tko je izdao najviše knjiga — gledaju se i kazne, oštećenja i gubici koji su nastali na posudbama koje je zaposlenik obradio.

```
SELECT

    zs.id_zaposlenik,

    zs.zaposlenik,

    zs.radno_mjesto,

    zs.datum_zaposlenja,

    SUM(zs.izdano_knjiga) AS ukupno_izdano,

    SUM(zs.posudbi_s_kaznom) AS ukupno_s_kaznom,

    SUM(zs.kazni_kasnjenje) AS kasnjenja,

    SUM(zs.kazni_ostecenje) AS ostecenja,

    SUM(zs.kazni_gubitak) AS gubici,

    SUM(zs.obracunato_kazni) AS ukupno_obracunato,

    SUM(zs.naplaceno_kazni) AS ukupno_naplaceno,

    MAX(zs.izdano_knjiga) AS najbolji_mjesec

FROM zaposlenici_statistika AS zs

WHERE zs.godina = '2025'

GROUP BY zs.id_zaposlenik

ORDER BY ukupno_izdano DESC;
```

Opis upita:

- U FROM dijelu podatke čitam iz pogleda zaposlenici_statistika pod aliasom zs, jer su sve potrebne vrijednosti za grupiranje i izračun već dostupne u tom pregledu.
- U WHERE dijelu filtriram samo podatke iz 2025. godine jer je ovo godišnji izvještaj. Godina je tekstualni stupac koji sam izvukla u pregledu sa DATE_FORMAT, pa uspoređujem s tekstom '2025'.
- U GROUP BY dijelu grupiram po zs.id_zaposlenik jer pogled daje podatke po mjesecima, a trebam ukupne godišnje brojeke za svakog zaposlenika. SUM

funkcije agregiraju mjesečne vrijednosti u godišnje ukupne.

- U ORDER BY dijelu sortirala sam od zaposlenika koji je izdao najviše knjiga prema dolje (DESC), što znači da su najaktivniji zaposlenici na početku izvještaja — oni koji imaju najviše izdanih knjiga godišnje.
- U SELECT dijelu navela sam sve potrebne attribute za evaluaciju: `zs.id_zaposlenik`, `zs.zaposlenik`, `zs.radno_mjesto` i `zs.datum_zaposlenja` identificiraju zaposlenika i njegovu poziciju.
Nakon toga prikazujem zbrojene podatke:
 - `SUM(zs.izdano_knjiga)` AS `ukupno_izdano` je zbroj izdanih knjiga kroz sve mjesece u godini
 - `SUM(zs.posudbi_s_kaznom)` AS `ukupno_s_kaznom` je koliko posudbi je završilo s kaznom
 - `SUM` za kasnjenja, ostecenja i gubici pokazuju koliko je kazni svakog tipa bilo.
 - `SUM(zs.obracunato_kazni)` je ukupan iznos obračunatih kazni,
 - `SUM(zs.naplaceno_kazni)` je koliko je od toga naplaćeno.
 - `MAX(zs.izdano_knjiga)` AS `najbolji_mjesec` prikazuje u kojem mjesecu je zaposlenik izdao najviše knjiga.

8.2.5 Pogled 3: kazne_pregled

Knjižnica treba pratiti trend kazni kroz vrijeme — raste li broj kazni, koji tipovi kazni su najčešći i koliko efikasno se naplaćuju dugovi. Ovaj view daje pojednostavljeni pregled svake kazne s osnovnim podacima koji su potrebni za mjesečnu analizu: koji je razlog kazne, koliki je iznos, kada je nastala i je li plaćena.

```
CREATE OR REPLACE VIEW kazne_pregled AS
SELECT
    rk.naziv_razloga AS razlog,
    ka.iznos,
    ka.datum,
    CASE WHEN ka.placeno = TRUE THEN 'Placeno' ELSE 'Neplaceno' END AS
status_placanja
FROM kazna AS ka
INNER JOIN razlog_kazne AS rk ON ka.id_razlog = rk.id_razlog;
```

Opis pogleda:

- U FROM dijelu podatke čitam iz tablice kazne sa aliasom ka jer mi je to polazna točka.
- Sa INNER JOIN naredbom povezala sam tablicu razlog_kazne pod aliasom rk jer trebam naziv razloga kazne. Koristila sam INNER JOIN jer svaka kazna mora imati razlog — kazna bez razloga je neispravan podatak. Uvjet spajanja je po atributu id_razlog iz obje tablice.
- U SELECT dijelu navela sam sve potrebne attribute: rk.naziv_razloga AS razlog prikazuje tekstualni naziv razloga kazne, ka.iznos je iznos kazne, ka.datum je datum kada je kazna evidentirana.
- CASE WHEN ka.placeno = TRUE THEN 'Placeno' ELSE 'Neplaceno' END AS status_placanja pretvara BOOLEAN vrijednost (TRUE ili FALSE) iz tablice kazna u tekstualne vrijednosti 'Placeno' ili 'Neplaceno' koje su čitljivije u izvještajima.
- Sa naredbom CREATE OR REPLACE VIEW kreirala sam pogled kazne_pregled kako bi se prikazale sve kazne s osnovnim podacima koji su potrebni za analizu trendova: koji je razlog kazne, koliki je iznos, kada je nastala i je li plaćena.

8.2.6 Select 3: Kronološki trend kazni po mjesecima

Voditelj knjižnice želi vidjeti kako se broj kazni razvija kroz mjesece. Raste li broj kašnjenja? Imamo li više gubitaka nego prije? Koliko novca od kazni ostaje nenaplaćeno? Ovaj izvještaj odgovara na ta pitanja grupirajući kazne po mjesecu i razdvajajući ih po tipu.

```
SELECT
    DATE_FORMAT(kp.datum, '%Y-%m') AS mjesec,
    SUM(CASE WHEN kp.razlog = 'Kasnjenje' THEN 1 ELSE 0 END) AS broj_kasnjenja,
    SUM(CASE WHEN kp.razlog = 'Ostecenje' THEN 1 ELSE 0 END) AS broj_ostecenja,
    SUM(CASE WHEN kp.razlog = 'Gubitak' THEN 1 ELSE 0 END) AS broj_gubitaka,
    COUNT(*) AS ukupno_kazni,
    SUM(kp.iznos) AS ukupni_iznos,
    SUM(CASE WHEN kp.status_placanja = 'Neplaceno' THEN kp.iznos ELSE 0 END) AS
neplaceni_iznos
FROM kazne_pregled AS kp
GROUP BY mjesec
```

```
ORDER BY mjesec;
```

Opis upita:

- U FROM dijelu podatke čitam iz pogleda kazne_pregled sa aliasom kp jer mi je to polazna točka.
- U GROUP BY grupiram rezultate po mjesecu te dobivam njihov zbroj za svaki mjesec
- U ORDER BY dijelu sam ih sortirala kronološki po mjesecu.
- U SELECT dijelu navela sam sve potrebne attribute:
DATE_FORMAT(kp.datum, '%Y-%m') AS mjesec izvlači godinu i mjesec iz datuma kazne kao tekst (npr. '2025-10'), što je osnova za grupiranje i prikaz trenda.
- SUM(CASE WHEN kp.razlog = 'Kasnjenje' THEN 1 ELSE 0 END) AS broj_kasnjenja broji koliko je kazni za kašnjenje nastalo taj mjesec — prolazi kroz svaki redak, ako je razlog 'Kasnjenje' broji 1, inače 0, pa SUM daje ukupan broj. Ista logika se primjenjuje za broj_ostecenja i broj_gubitaka kako bih razdvojila kazne po tipu.
- COUNT(*) AS ukupno_kazni broji sve kazne u tom mjesecu bez obzira na razlog, SUM(kp.iznos) AS ukupni_iznos zbrajanja ukupan iznos svih kazni u eurima za taj mjesec, a SUM(CASE WHEN kp.status_placanja = 'Neplaceno'

8.3 Kelvin Jurković

Modul obuhvaća relacije CLAN, POSUDBA, REZERVACIJA, KNJIGA, PRIMJERAK i STATUS_PRIMJERKA. Služi za praćenje aktivnosti članova knjižnice, njihovih posudbi, rezervacija knjiga i dostupnosti pojedinih primjeraka.

8.3.1 Pogled 1: statistika_clanova

Statistika članova

Statistika posudbi za svakog člana knjižnice

Praktični poslovni primjer

Zaposleniku knjižnice je u svakodnevnom radu knjižnice korisno imati brz način za provjeriti aktivnost članova, tj. vidjeti podatke o članu i koliko je svaki član imao ukupnih posudbi, aktivnih posudbi i da li možda postoje posudbe koje kasne sa vraćanjem. Kako ne bi trebao svaku posudbu ručno pregledavati, napravili smo pogled s kojim objedinjujemo podatke o članovima i njihovim posudbama.

Razlog izrade pogleda

Pogled je napravljen kako bi nam olakšao izradu nadolazećeg 'Upit 1' tako da imamo već pripremljene podatke i da sa njima možemo pronaći članove koji trenutno imaju više aktivnih posudbi od prosjeka

TRAŽENI PODACI

id_clan, ime, prezime, email, status_clana, ukupan_broj_posudbi, broj_aktivnih_posudbi, broj_vracenih_posudbi, broj_zakasnjelih_vracanja, trenutno_zakasnjele_posudbe, zadnja_posudba

KOD

```
CREATE VIEW statistika_clanova AS
SELECT
    c.id_clan,
    c.ime,
    c.prezime,
    c.email,

    c.status AS status_clana,
    COUNT(p.id_posudba) AS ukupan_broj_posudbi,
```

```

SUM(
    CASE
        WHEN p.status = 'Aktivno' THEN 1
        ELSE 0
    END
) AS broj_aktivnih_posudbi,

SUM(
    CASE
        WHEN p.status = 'Vraceno' then 1
        ELSE 0
    end
) AS broj_vracenih_posudbi,

SUM(
    CASE
        WHEN p.datum_vracanja IS NOT NULL AND p.datum_vracanja >
p.rok_vracanja THEN 1
        ELSE 0
    END
) AS broj_zakasnjelih_vracanja,

SUM(
    CASE
        WHEN p.status = 'Aktivno' AND p.rok_vracanja < CURDATE()
        THEN 1
        ELSE 0
    END
) AS trenutno_zakasnjele_posudbe,

MAX(p.datum_posudbe) AS zadnja_posudba
FROM clan AS c
LEFT JOIN posudba AS p
    ON c.id_clan = p.id_clan
GROUP BY
    c.id_clan,
    c.ime,
    c.prezime,
    c.email,
    c.status;

```

OPIS POGLEDA

- **FROM clan AS c** - krećemo od tablice clan jer pogled koristimo za članove knjižnice i želimo za svakog člana knjižnice dobiti njegovu statistiku posudbi
- **LEFT JOIN posudba AS p** - povezujemo tablicu clan sa tablicom posudba kako bi za svakog člana mogli dohvatiti njegove posudbe sa ovom naredbom kako bi se u rezultatu prikazali i članovi koji još nisu ni napravili posudbu
- **ON c.id_clan = p.id_clan** - koristimo ON uvjet da povežemo člana sa njegovim posudbama preko atributa id_clan
- **GROUP BY** - grupiramo podatke po članu jer jedan član može imati više posudbi a želimo da se u pogledu svaki član prikaže samo jednom
- **SELECT** - biramo podatke koje želimo prikazati u pogledu, specifično želimo osnovne podatke (id, ime, prezime, email, status) i vrijednosti koje ćemo izračunati kao broj aktivnih, vraćenih, i zakašnjelih posudbi i broj broj zakašnjelih vraćanja
- **COUNT(p.id_posudba) AS ukupan_broj_posudbi** - koristimo funkciju agregacije da izračunamo ukupan broj posudbi za svakog člana
- **SUM(CASE WHEN ... THEN 1 ELSE 0 END)** - koristimo funkciju agregacije SUM zajedno sa uvjetnim izrazom CASE WHEN kako bi prebrojali aktivno posudbe (kada je p.status 'Aktivno'), broj vraćenih posudba (kada je p.status 'Vraceno'), broj zakašnjelih vraćanja (kada p.datum_vracanja nije null i kada je veći od p.rok_vracanja) i broj trenutno zakašnjelih posudbi (kada je p.status 'Aktivno' i p.rok_vracanja manji od današnjeg datuma)
- **MAX(p.datum_posudbe) AS zadnja_posudba** - koristimo funkciju agregacije za vraćanje maksimalne vrijednosti da dobijemo datum zadnje posudbe pojedinog člana
- **CREATE VIEW statistika_clanova AS** - rezultat cijele SELECT naredbe koje smo napravili spremamo kao pogled statistika_clanova kako bi se mogao kasnije koristiti

8.3.2 Pogled 2: aktivne_posudbe_detalji

Aktivne posudbe detalji

Detaljni prikaz svih aktivnih posudbi

Praktični poslovni primjer

Zaposleniku knjižnice je u svakodnevnom radu knjižnice korisno imati pripremljen prikaz svih trenutno aktivnih posudbi i njihovih detalja da na jednom mjestu mogu vidjeti koje se knjige nalaze kod članova ili koji su primjerci posuđeni i kada se moraju

vratiti. Kako ne bi trebao posebno ručno uspoređivati i pregledavati posudbe, članove, knjige i primjerke napravili smo pogled s kojim spajamo sve podatke iz tih tablica u jedan pregled.

Razlog izrade pogleda

Pogled je napravljen kako bi nam olakšao izradu nadolazećeg 'Upit 2' tako da imamo već pripremljene podatke o posudbi, članu, knjizi, primjerku i stanju roka vraćanja tako da bi prikazali aktivne posudbe ili posudbe koje uskoro ističu.

TRAŽENI PODACI

id_posudba, id_clan, ime_clana, prezime_clana, id_knjiga, naslov, id_primjerak, inventarni_broj, datum_posudbe, rok_vracanja, dani_nakon_roka, dana_do_roka, stanje_roka

KOD

```
CREATE VIEW aktivne_posudbe_detalji AS
SELECT
    p.id_posudba,
    c.id_clan,
    c.ime AS ime_clana,
    c.prezime AS prezime_clana,
    k.id_knjiga,
    k.naslov,
    pr.id_primjerak,
    pr.inventarni_broj,
    p.datum_posudbe,
    p.rok_vracanja,

    -- pozitivna vrijednost -> rok je prosao
    DATEDIFF(CURDATE(), p.rok_vracanja) AS dani_nakon_roka,

    -- pozitivna vrijednost -> koliko dana preostaje do roka
    DATEDIFF(p.rok_vracanja, CURDATE()) AS dana_do_roka,

    CASE
        WHEN p.rok_vracanja < CURDATE() THEN 'Kasni'
        ELSE 'U roku'
    END AS stanje_roka
FROM posudba AS p
INNER JOIN clan AS c
    ON p.id_clan = c.id_clan
```

```

INNER JOIN primjerak as pr
  ON p.id_primjerak = pr.id_primjerak
INNER JOIN knjiga as k
  ON pr.id_knjiga = k.id_knjiga
WHERE p.status = 'Aktivno';

```

OPIS POGLEDA

- **FROM posudba AS p** - krećemo od tablice posudba jer pogled koristimo za prikaz detalja posudba
- **INNER JOIN clan as c** - povezujemo tablicu posudba sa tablicom clan sa INNER JOIN jer nam svaka posudba treba imati člana kojemu pripada i također u rezultatu želimo prikazati podatke o tom članu
- **ON p.id_clan = c.id_clan** - koristimo ON uvjet da odredimo po kojem atributu povezujemo tablice, tj. povezujemo posudbu sa članom preko atributa id_clan
- **INNER JOIN primjerak as pr** - povezujemo tablicu posudba sa tablicom primjerak sa INNER JOIN jer nam svaka posudba treba imati primjerak knjige
- **ON p.id_primjerak = pr.id_primjerak** - koristimo ON uvjet da odredimo koji je primjerak vezan uz pojedinu posudbu stoga povezujemo posudbu sa primjerkom preko atributa id_primjerak
- **INNER JOIN knjiga AS k** - povezujemo tablicu knjiga sa tablicom primjerak sa INNER JOIN jer nam svaki primjerak pripada određenoj knjizi i preko nje možemo i dohvatiti podatke o knjizi
- **ON pr.id_knjiga = k.id_knjiga** - koristimo ON uvjet da odredimo kojoj knjizi pripada pojedini primjerak stoga povezujemo primjerak sa knjiga preko atributa id_knjiga
- **WHERE p.status = 'Aktivno'** - odabiremo samo posudbe koje su trenutno aktivne
- **SELECT** - biramo podatke koje želimo prikazati u pogledu, tj. podatke o posudbi, članu, knjizi, primjercima i rokovima vraćanja
- **DATEDIFF(CURDATE(), p.rok_vracanja) AS dani_nakon_roka** - koristimo funkciju DATEDIFF da bi izračunali koliko je dana prošlo od vraćanja roka, današnji datum dobivamo sa CURDATE()
- **DATEDIFF(p.rok_vracanja, CURDATE()) AS dana_do_roka** - koristimo funkciju DATEDIFF da bi izračunali koliko dana preostaje do roka vraćanja, današnji datum također dobivamo sa CURDATE()
- **CASE WHEN...THEN...ELSE...END AS stanje_roka** - koristimo CASE WHEN da bi odredili stanje roka tako da gledamo ako je rok vraćanja manji od današnjeg datuma (p.rok_vracanja < CURDATE()) onda posudba dobiva oznaku 'Kasni', a ako ne onda dobiva oznaku 'U roku'

- **CREATE VIEW aktivne_posudbe_detalji AS** - rezultat cijele SELECT naredbe koju smo napravili spremamo kao pogled aktivne_posudbe_detalji kako bi se mogao kasnije koristiti

8.3.3 Pogled 3: rezervacije_dostupnost

Rezervacije dostupnost

Prikaz rezervacija zajedno sa dostupnošću primjeraka knjige

Praktični poslovni primjer

Zaposleniku knjižnice je u svakodnevnom radu knjižnice korisno imati pripremljeni prikaz rezervacija knjiga i njihovu dostupnost primjeraka, na taj način zaposlenik može na jednom mjestu vidjeti tko je napravio rezervaciju, za koju knjigu, koliko ta knjiga ima primjeraka, koliko ih je dostupno i koliko ih je trenutno posuđeno. Kako ne bi trebao posebno ručno pregledavati rezervacije, knjige, primjerke, statuse primjeraka i posudbe napravili smo pogled s kojim spajamo sve te podatke u jedan pregled.

Razlog izrade pogleda

Pogled je napravljen kako bi nam olakšao izradu nadolazećeg 'Upit 3' tako da već imamo pripremljene podatke o rezervacijama i dostupnostima primjeraka kako bi prikazali knjige koje imaju više aktivnih rezervacija nego dostupnih primjeraka

Traženi podaci

id_rezervacija, id_clan, ime_clana, prezime_clana, id_knjiga,
naslov, datum_rezervacije, status_rezervacije,
ukupan_broj_primjeraka, broj_dostupnih_primjeraka,
broj_aktivno_posuđenih_primjeraka

KOD

```
CREATE VIEW rezervacije_dostupnost AS
SELECT
    r.id_rezervacija,
    c.id_clan,
    c.ime AS ime_clana,
    c.prezime AS prezime_clana,
    k.id_knjiga,
    k.naslov,
    r.datum_rezervacije,
```

```

r.status_rezervacije,

COUNT(DISTINCT pr.id_primjerak) AS ukupan_broj_primjeraka,

COUNT (
    DISTINCT CASE
        WHEN sp.naziv_statusa = 'Dostupno'
        THEN pr.id_primjerak
    END
) AS broj_dostupnih_primjeraka,

COUNT (
    DISTINCT CASE
        WHEN p.status = 'Aktivno'
        THEN pr.id_primjerak
    END
) AS broj_aktivno_posudjenih_primjeraka

FROM rezervacija as r
INNER JOIN clan as c
    ON r.id_clan = c.id_clan
INNER JOIN knjiga as k
    ON r.id_knjiga = k.id_knjiga
LEFT JOIN primjerak as pr
    ON k.id_knjiga = pr.id_knjiga
LEFT JOIN status_primjerka AS sp
    ON pr.id_status = sp.id_status
LEFT JOIN posudba as p
    ON pr.id_primjerak = p.id_primjerak
    AND p.status = 'Aktivno'
GROUP BY
    r.id_rezervacija,
    c.id_clan,
    c.ime,
    c.prezime,
    k.id_knjiga,
    k.naslov,
    r.datum_rezervacije,
    r.status_rezervacije;

```

OPIS POGLEDA

- **FROM rezervacija as r** - krećemo od tablice rezervacija jer pogled koristimo za prikaz rezervacije knjiga i njihovu povezanost sa dostupnim primjercima
- **INNER JOIN clan AS c** - tablicu clan spajamo sa tablicom rezervacija pomoću INNER JOIN jer nam svaka rezervacija treba imati člana koji ju je napravio
- **ON r.id_clan = c.id_clan** - koristimo ON uvjet da odredimo da se rezervacija povezuje sa članom preko atributa id_clan
- **INNER JOIN knjiga AS k** - tablicu knjiga povezujemo s tablicom rezervacija pomoću INNER JOIN jer nam svaka rezervacija treba biti vezana uz određenu knjigu
- **ON r.id_knjiga = k.id_knjiga** - koristimo ON uvjet da odredimo na koju se knjigu odnosi pojedina rezervacija preko atributa id_knjiga
- **LEFT JOIN primjerak as pr** - tablicu primjerak povezujemo sa tablicom knjiga pomoću LEFT JOIN kako bi rezervacija i knjiga ostale u rezultatu i u slučaju kada za knjigu nema povezanih primjeraka
- **ON k.id_knjiga = pr.id_knjiga** - koristimo ON uvjet da odredimo koji primjerci pripadaju kojoj pojedinoj knjizi pomoću atributa id_knjiga
- **LEFT JOIN status_primjerka AS sp** - tablicu status_primjerka povezujemo sa tablicom primjerak pomoću LEFT JOIN kako bi mogli provjeriti status svakog primjerka da kasnije s njime možemo računati koliko je primjeraka knjige dostupno
- **ON pr.id_status = sp.id_status** - koristeći ON uvjet da povežemo primjerak sa njegovim statusom preko atributa id_status
- **LEFT JOIN posudba AS p** - tablicu posudba povezujemo sa tablicom primjerak pomoću LEFT JOIN da bi provjerili koji primjerci su trenutno aktivno posuđeni jer želimo zadržati sve primjerke knjige u rezultatu čak i one bez aktivne posudbe
- **ON pr.id_primjerak = p.id_primjerak AND p.status = 'Aktivno'** - koristeći ON uvjet povezujemo primjerak sa njegovom aktivnom posudbom, uvjet p.status = 'Aktivno' stavljen je da bi se gledale samo aktivne posudbe a da i primjerci bez aktivne posudbe zbog LEFT JOIN ostanu i dalje u rezultatu
- **GROUP BY** - grupiramo podatke po rezervaciji, članu, i knjizi jer jedna knjiga može imati više primjeraka a u pogledu želimo dobiti po jedan red za svaku rezervaciju
- **SELECT** - biramo podatke koje želimo prikazati u pogledu, tj. podatke o rezervaciji, članu, knjizi i vrijednosti o primjercima koje ćemo izračunati
- **COUNT(DISTINCT pr.id_primjerak) AS ukupan_broj_primjeraka** - koristimo funkciju agregacije COUNT da bi prebrojali ukupan broj primjeraka za pojedinu knjigu i pritom koristimo DISTINCT da bi se isti primjerak brojao samo jednom
- **COUNT(DISTINCT CASE WHEN sp.naziv_statusa = 'Dostupno' THEN pr.id_primjerak END) AS broj_dostupnih_primjeraka** - koristimo funkciju agregacije COUNT i uvjetni izraz CASE WHEN da bi

prebrojali sve primjerke koji imaju status dostupno (sp.naziv_statusa = 'Dostupno') i koristimo DISTINCT da bi se svaki dostupni primjerak brojao samo jednom

- **COUNT(DISTINCT CASE WHEN p.status = 'Aktivno' THEN pr.id_primjerak END) AS broj_aktivno_posudjenih_primjeraka** - koristimo funkciju agregacije COUNT i uvijetni izraz CASE WHEN da bi prebrojali primjerke koji su aktivno posuđeni i koristimo DISTINCT da bi se ne bi svako posuđeni primjerak brojao više puta
- **CREATE VIEW rezervacije_dostupnost AS** - rezultat cijele SELECT naredbe spremamo kao pogled rezervacije_dostupnost za daljnje korištenje

8.3.4 Upit 1: Članovi s iznadprosječnim brojem aktivnih posudbi

Članovi s iznadprosječnim brojem aktivnih posudbi

Prikaz članova koji trenutno imaju više aktivnih posudbi od prosjeka

Praktični poslovni primjer

Knjižničarka Nina Radić je dobila zadatak da pripremi popis članova koji imaju više aktivnih posudbi od prosjeka. Razlog je taj jer knjižnica želi dodatno provjeriti njihove posudbe i na vrijeme ih podsjetiti na obvezu vraćanja u svrhu smanjenja mogućnosti zakašnjelih vraćanja i lakšeg praćenja članova sa većim brojem aktivnih posudbi od prosjeka.

Koristi pogled

Upit koristi prethodno napravljeni Pogled 1 statistika_clanova jer smo u njemu već pripremili podatke o članovima i njihovim posudbama. Iz pogleda već možemo dohvatiti većinu podataka i jedino moramo izdvojiti članove koji imaju više aktivnih posudbi od prosjeka.

Traženi rezultati

id_clan, ime, prezime, email, status_clana, ukupan_broj_posudbi, broj_aktivnih_posudbi, broj_vracenih_posudbi, broj_zakasnjelih_vracanja, trenutno_zakasnjele_posudbe, zadnja_posudba

KOD

```
SELECT
    sc.id_clan,
    sc.ime,
    sc.prezime,
    sc.email,
    sc.status_clana,
    sc.ukupan_broj_posudbi,
    sc.broj_aktivnih_posudbi,
    sc.broj_vracenih_posudbi,
    sc.broj_zakasnjelih_vracanja,
    sc.trenutno_zakasnjele_posudbe,
    sc.zadnja_posudba

FROM statistika_clanova as sc
WHERE sc.broj_aktivnih_posudbi > (
    SELECT AVG(broj_aktivnih_posudbi)
    FROM statistika_clanova
)
ORDER BY
    sc.broj_aktivnih_posudbi DESC,
    sc.trenutno_zakasnjele_posudbe DESC,
    sc.prezime ASC;
```

OPIS UPITA

- **FROM statistika_clanova AS sc** - koristimo pogled statistika_clanova jer smo u njemu već pripremili podatke o članovima i njihovim posudbama
- **WHERE sc.broj_aktivnih_posudbi > (...)** - želimo prikazati samo one članove koji imaju više aktivnih posudbi od prosjeka
- **(SELECT AVG(broj_aktivnih_posudbi) FROM statistika_clanova)** - u WHERE dijelu koristimo podupit da izračunamo prosječan broj aktivnih posudbi korištenjem funkcije agregacije AVG i dobiveni prosjek uspoređujemo sa brojem aktivnih posudbi svakog člana iz vanjskog upita
- **SELECT** - biramo podatke za koje želimo prikazati u rezultatu, što su osnovni podaci o članu i izračunate vrijednosti iz pogleda statistika_clanova
- **ORDER BY sc.broj_aktivnih_posudbi DESC** - rezultati sortiramo silazno po broju aktivnih posudbi tako da se članovi sa najvećim brojem aktivnih posudbi prikažu prvi

- **sc.trenutno_zakasnjele_posudbe DESC** - ako nam više članova ima isti broj aktivnih posudbi, prvo prikazujemo članove koji imaju više trenutni aktivnih zakašnjelih posudbi
- **sc.prezime ASC** - na kraju rezultat sortiramo po prezimenu člana da nam bude pregledniji prikaz

– PLACEHOLDER REZULTAT ?? –

8.3.5 Upit 2: Aktivne posudbe koje kasne ili uskoro ističu

Aktivne posudbe koje kasne ili uskoro ističu

Prikaz aktivnih posudbi koje zahtijevaju pažnju zaposlenika jer kasne ili im rok vraćanja ističe unutar 5 dana.

Praktični poslovni primjer

Knjižničarka Nina Radić je odlučila provjeriti aktivne posudbe kojima je istekao rok vraćanja ili im unutar sljedećih pet dana ističe. Namjerava na vrijeme kontaktirati članove kako bi ih podsjetila na vraćanje knjiga i smanjila broj zakašnjelih vraćanja ili ih podsjetila u slučaju da su zaboravili na rok vraćanja knjige. Dodano je namjeravala vidjeti postoji li i aktivna rezervacija za istu knjigu jer možda drugi član čeka da se vrati.

Korišteni pogled

Upit koristi prethodno napravljen Pogled 2 aktivne_posudbe_detalji jer smo u njemu već pripremili podatke o aktivnim posudbama, članovima, knjigama, primjercima i rokovima vraćanja te iz njega možemo dohvatiti većinu potrebnih podataka, a u ovom upitu ćemo izdovijiti posudbe koje kasne ili kojima rok ističe unutar pet dana.

TRAŽENI REZULTATI

id_posudba, id_clan, ime_clana, prezime_clana, id_knjiga, naslov, id_primjerak, inventarni_broj, datum_posudbe, rok_vracanja, dani_nakon_roka, dana_do_roka, stanje_roka, broj_aktivnih_rezervacija_za_knjigu

KOD

```
SELECT
    apd.id_posudba,
    apd.id_clan,
```

```

    apd.ime_clana,
    apd.prezime_clana,
    apd.id_knjiga,
    apd.naslov,
    apd.id_primjerak,
    apd.inventarni_broj,
    apd.datum_posudbe,
    apd.rok_vracanja,
    apd.dani_nakon_roka,
    apd.dana_do_roka,
    apd.stanje_roka,
    (
        SELECT COUNT(*)
        FROM rezervacija as r
        WHERE r.id_knjiga = apd.id_knjiga AND r.status_rezervacije =
'Aktivna'
    ) AS broj_aktivnih_rezervacija_za_knjigu

FROM aktivne_posudbe_detalji as apd
WHERE apd.stanje_roka = 'Kasni' OR apd.dana_do_roka BETWEEN 0 AND 5
ORDER BY
    apd.stanje_roka ASC,
    apd.rok_vracanja ASC;

```

OPIS UPITA

- **FROM aktivne_posudbe_detalji AS apd** - koristimo pogled aktivne_posudbe_detalji jer smo u njemu već pripremili sve detalje o aktivnim posudbama
- **WHERE apd.stanje_roka = 'Kasni' OR apd.dana_do_roka BETWEEN 0 AND 5** - koristimo selekciju da prikazemo samo posudbe koje kasne (apd.stanje_roka = 'Kasni') ili (OR) kojima rok vraćanja ističe unutar 5 dana (apd.dana_do_roka BETWEEN 0 AND 5)
- **SELECT** - biramo podatke koje želimo prikazati u rezultatu što bi bili podaci o posudbi, članu, knjizi, primjerku, rokovima vraćanja i stanju roka
- **(SELECT COUNT(*) FROM rezervacija as r WHERE r.id_knjiga = apd.id_knjiga AND r.status_rezervacije = 'Aktivna') AS broj_aktivnih_rezervacija_za_knjigu** - u SELECT dijelu koristimo kao podupit da bi izračunali broj aktivnih rezervacija za istu knjigu i to radimo funkcijom agregacije COUNT(*) kojom prebrojavamo ukupan broj redova koji zadovoljavaju uvjete u podupitu, uvjetom r.id_knjiga = apd.id_knjiga

povezujemo rezervacije sa knjigom iz vanjskog upita a uvjetom `r.status_rezervacije = 'Aktivna'` brojimo samo aktivne rezervacije

- **ORDER BY `apd.stanje_roka ASC`** - rezultat sortiramo prema stanju roka uzlazno za pregledniji prikaz
- **`apd.rok_vracanja ASC`** - nakon toga rezultat sortiramo uzlazno prema roku vraćanja tako da se prvo prikažu posudbe s najranijim rokom vraćanja

– PLACEHOLDER REZULTAT ?? –

8.3.6 Upit 3: Knjige s više aktivnih rezervacija nego dostupnih primjeraka

Knjige s više aktivnih rezervacija nego dostupnih primjeraka

Prikaz knjiga za koje je potražnja veća nego trenutna dostupnost

Praktični poslovni primjer

Knjižničarka Nina Radić je dobila zadatak da provjeri za koje knjige postoji veći broj aktivnih rezervacija nego dostupnih primjeraka da bi knjižnica na temelju toga mogla vidjeti za koje knjige postoji veća potražnja od trenutne dostupnosti i za koje knjige nema adekvatna količina dostupnih primjeraka. Taj popis je potreban za donošenje odluke treba li nabaviti dodatne primjerke nekih knjiga i za obraćanje pažnje za primjerke knjiga koje se vraćaju da se ponovno oslobode.

Korišteni pogled

Upit koristi prethodno napravljeni Pogled 3 `rezervacije_dostupnost` jer smo u njemu već pripremili podatke o rezervacijama, knjigama i dostupnosti primjeraka te iz njega već možemo dohvatiti podatke o rezervacijama, njegovom statusu i broju dostupnih primjeraka, a u ovom upitu dodatno ćemo izdvojiti samo one knjige kod kojih je broj aktivnih rezervacija veći od broja dostupnih primjeraka.

Traženi rezultat

`id_knjiga`, `naslov`, `broj_aktivnih_rezervacija`, `broj_dostupnih_primjeraka`, `manjak_primjeraka`

KOD

```

SELECT
    rd.id_knjiga,
    rd.naslov,
    COUNT(rd.id_rezervacija) AS broj_aktivnih_rezervacija,
    MAX(rd.broj_dostupnih_primjeraka) AS broj_dostupnih_primjeraka,
    COUNT(rd.id_rezervacija) - MAX(rd.broj_dostupnih_primjeraka) AS
manjak_primjeraka

FROM rezervacije_dostupnost AS rd
WHERE rd.status_rezervacije = 'Aktivna'
GROUP BY
    rd.id_knjiga,
    rd.naslov
HAVING COUNT(rd.id_rezervacija) > MAX(rd.broj_dostupnih_primjeraka)
ORDER BY manjak_primjeraka DESC;

```

OPIS UPITA

- **FROM rezervacije_dostupnost AS rd** - koristimo pogled rezervacije_dostupnost jer smo u jemu već pripremili podatke o rezervacijama i dostupnosti primjeraka knjige
- **WHERE rd.status_rezervacije = 'Aktivna'** - odabiremo samo aktivne rezervacije
- **GROUP BY rd.id_knjiga, rd.naslov** - grupiramo podatke po knjizi jer jedna knjiga može imati više rezervacija, a u rezultatu želimo za svaku knjigu jedan red
- **HAVING COUNT(rd.id_rezervacija) > MAX(rd.broj_dostupnih_primjeraka)** - koristimo filtriranje nakon grupiranja da bi prikazali samo one knjige kod kojih je broj aktivnih rezervacija veći od broja dostupnih primjeraka, sa funkcijom agregacije COUNT(rd.id_rezervacija) računamo broj aktivnih rezervacija za pojedinu knjigu jer su prije toga u WHERE dijelu već odabrane samo aktivne rezervacije, funkcijom agregacije MAX(rd.broj_dostupnih_primjeraka) dohvaćamo broj dostupnih primjeraka za tu knjigu
- **SELECT** - biramo podatke koje želimo prikazati u rezultatu što su id knjige, naslov knjige i izračunate vrijednosti vezane za rezervacije i dostupne primjerke
- **COUNT(rd.id_rezervacija) AS broj_aktivnih_rezervacija** - koristimo funkciju agregacije COUNT(rd.id_rezervacija) da bi prebrojali broj aktivnih rezervacija za svaku knjigu
- **MAX(rd.broj_dostupnih_primjeraka) AS broj_dostupnih_primjeraka** - koristimo funkciju agregacije MAX () kako bi se iz grupe dobija vrijednost broja dostupnih primjeraka za tu knjigu

- **COUNT(rd.id_rezervacija) -**
MAX(rd.broj_dostupnih_primjeraka) AS manjak_primjeraka - računamo koliko primjeraka nedostaje tako da od broja aktivnih rezervacija oduzmemo broj dostupnih primjeraka
- **ORDER BY manjak_primjeraka DESC** - rezultat sortiramo sliazno prema manjku primjeraka da nam se prvo prikažu knjige kojima nedostaje najviše primjeraka

– PLACEHOLDER REZULTAT ?? –

8.4 Vice Jukić

Modul Klasifikacija i fond obuhvaća relacije KNJIGA, KNJIGA _ ZANR, ZANR, PRIMJERAK, STATUS _ PRIMJERKA i LOKACIJA. Služi za organizaciju i praćenje knjižničkog fonda, a omogućuje pregled žanrova, primjeraka, njihovih lokacija i statusa dostupnosti knjiga.

8.4.1. Pogled 1: analiza_nedostupnih_primjeraka

--Analiza odjela s povećanim brojem nedostupnih primjeraka--

U knjižnici se dio primjeraka knjiga trenutno ne može koristiti zbog posudbe, oštećenja, restauracije ili drugih razloga. Zbog toga je potrebno analizirati u kojim odjelima knjižnice postoji natprosječni broj nedostupnih primjeraka te koji su njihovi statusi.

Kako bi se omogućila takva analiza, sustav prvo mora omogućiti pregled svih nedostupnih primjeraka zajedno s naslovom knjige, inventarnim brojem, statusom i lokacijom primjerka.

TRAŽENO RJEŠENJE:

odjel, kat, status _primjerka, broj _nedostupnih _primjeraka

```
CREATE OR REPLACE VIEW analiza_nedostupnih_primjeraka AS
SELECT  p.id_primjerak,
        p.inventarni_broj,
        k.naslov,
        sp.naziv AS status_primjerka,
        sp.opis AS opis_statusa,
        l.odjel,
        l.polica,
        l.kat
FROM    primjerak AS p
INNER JOIN knjiga AS k
        ON p.id_knjiga = k.id_knjiga
INNER JOIN status_primjerka AS sp
        ON p.id_status = sp.id_status
INNER JOIN lokacija AS l
        ON p.id_lokacija = l.id_lokacija
WHERE   sp.dostupan = FALSE;
```


Opis pogleda

- FROM primjerak AS p - koristim tablicu primjerak kao glavnu tablicu jer ona sadrži osnovne podatke o svakom primjerku knjige koji se nalazi u knjižničnom fondu.
- INNER JOIN knjiga AS k ON p.id_knjiga = k.id_knjiga - tablicu knjiga povezujem s tablicom primjerak preko atributa id_knjiga kako bih za svaki primjerak mogao prikazati naslov pripadajuće knjige.
- INNER JOIN status_primjerka AS sp ON p.id_status = sp.id_status - tablicu status_primjerka povezujem preko atributa id_status kako bih mogao prikazati naziv i opis statusa svakog primjerka knjige.
- INNER JOIN lokacija AS l ON p.id_lokacija = l.id_lokacija - tablicu lokacija povezujem preko atributa id_lokacija kako bih mogao prikazati odjel, policu i kat na kojem se primjerak nalazi.
- WHERE sp.dostupan = FALSE - koristim uvjet kojim izdvajam samo one primjerke koji trenutno nisu dostupni korisnicima knjižnice.
- SELECT - u SELECT dijelu biram podatke koje želim prikazati u rezultatu pregleda, a to su id primjerka, inventarni broj, naslov knjige, naziv i opis statusa primjerka te podaci o lokaciji primjerka.
- sp.naziv AS status_primjerka - atribut naziv iz tablice status_primjerka preimenovao sam u status_primjerka kako bi naziv atributa bio jasniji u rezultatu pregleda.
- sp.opis AS opis_statusa - atribut opis iz tablice status_primjerka preimenovao sam u opis_statusa radi preglednijeg prikaza podataka.
- CREATE OR REPLACE VIEW analiza_nedostupnih_primjeraka - pomoću naredbe CREATE OR REPLACE VIEW kreirao sam pregled pod nazivom analiza_nedostupnih_primjeraka koji služi za daljnju analizu nedostupnih primjeraka u knjižničnom fondu.

8.4.2. Upit 1: Analiza odjela s povećanim brojem nedostupnih primjeraka

```
SELECT  anp.odjel,
        anp.kat,
        anp.status_primjerka,
        COUNT(anp.id_primjerak) AS broj_nedostupnih_primjeraka
FROM    analiza_nedostupnih_primjeraka AS anp
WHERE   anp.kat != 'Skladiste'
```

```

GROUP BY anp.odjel, anp.kat, anp.status_primjerka
HAVING COUNT(anp.id_primjerak) >
(
    SELECT AVG(broj_nedostupnih)
    FROM (
        SELECT COUNT(anp2.id_primjerak) AS broj_nedostupnih
        FROM analiza_nedostupnih_primjeraka AS anp2
        WHERE anp2.kat != 'Skladiste'
        GROUP BY anp2.odjel, anp2.status_primjerka
    ) AS prosjek_nedostupnih
)
ORDER BY broj_nedostupnih_primjeraka DESC,
        anp.odjel ASC,
        anp.status_primjerka ASC;

```

Opis upita

- FROM analiza_nedostupnih_primjeraka AS anp - koristim pregled analiza_nedostupnih_primjeraka jer su u njemu već pripremljeni svi podaci o nedostupnim primjercima, njihovim statusima i lokacijama.
- SELECT - u SELECT dijelu biram podatke koje želim prikazati u rezultatu upita, a to su odjel, kat, status primjerka i broj nedostupnih primjeraka.
- COUNT(anp.id_primjerak) AS broj_nedostupnih_primjeraka - pomoću funkcije agregacije COUNT računam broj nedostupnih primjeraka u svakom odjelu te rezultat preimenujem u broj_nedostupnih_primjeraka.
- WHERE anp.kat != 'Skladiste' - koristim uvjet kojim isključujem primjerke koji se nalaze u skladištu kako analiza ne bi uključivala primjerke koji nisu dostupni korisnicima na policama knjižnice.
- GROUP BY anp.odjel, anp.kat, anp.status_primjerka - podatke grupiram prema odjelu, katu i statusu primjerka kako bih za svaku kombinaciju mogao izračunati broj nedostupnih primjeraka.
- HAVING COUNT(anp.id_primjerak) > (...) - koristim HAVING dio upita kako bih izdvojio samo one odjele i status primjeraka koji imaju natprosječan broj nedostupnih primjeraka.
- SELECT AVG(broj_nedostupnih) FROM (...) AS prosjek_nedostupnih - pomoću podupita računam prosječan broj nedostupnih primjeraka svih odjela i statusa primjeraka.
- SELECT COUNT(anp2.id_primjerak) AS broj_nedostupnih - u unutarnjem podupitu prvo računam broj nedostupnih primjeraka po odjelima i statusima kako bih kasnije mogao izračunati njihov prosjek.
- ORDER BY broj_nedostupnih_primjeraka DESC, anp.odjel ASC, anp.status_primjerka ASC - rezultate sortiram silazno prema broju nedostupnih primjeraka, a zatim uzlazno prema nazivu odjela i statusu primjerka radi preglednijeg prikaza rezultata.

--rezultat tablica(?)--

8.4.3. Pogled 2: fond_po_odjelu

-- Analiza najzastupljenijih odjela knjižničkog fonda --

U knjižnici pojedini odjeli sadrže znatno veći broj primjeraka knjiga od ostalih odjela zbog čega zauzimaju više prostora i otežavaju organizaciju knjižničkog fonda. Zbog toga je potrebno analizirati koji odjeli imaju natprosječan broj primjeraka u odnosu na ostatak knjižnice.

Kako bi se omogućila takva analiza, sutav prvo mora omogućiti pregled broja primjeraka po lokacijama i odjelima knjižnice.

TRAŽENO RJEŠENJE:

odjel, kat, ukupan _broj _primjeraka, prosjecan _broj _primjeraka, najveći _broj _primjeraka _na _lokaciji, najmanji _broj _primjeraka _na _lokaciji

```
CREATE OR REPLACE VIEW fond_po_odjelu AS
SELECT  l.id_lokacija,
        l.odjel,
        l.kat,
        COUNT(p.id_primjerak) AS broj_primjeraka
FROM    lokacija AS l
LEFT JOIN primjerak AS p
        ON l.id_lokacija = p.id_lokacija
GROUP BY l.id_lokacija, l.odjel, l.kat;
```

Opis pogleda

- FROM lokacija AS l - koristim tablicu lokacija kao glavnu tablicu jer želim prikazati sve odjele i lokacije unutar knjižnice.
- LEFT JOIN primjerak AS p ON l.id_lokacija = p.id_lokacija - tablicu primjerak povezujem s tablicom lokacija preko atributa id_lokacija kako bih mogao dohvatiti primjerke koji se nalaze na pojedinoj lokaciji.
- LEFT JOIN koristim kako bi se u rezultatu prikazale i lokacije koje trenutno nemaju nijedan primjerak knjige.
- SELECT - u SELECT dijelu biram podatke koje želim prikazati u rezultatu pregleda, a to su identifikator lokacije, naziv odjela, kat i broj primjeraka na pojedinoj lokaciji.
- COUNT(p.id_primjerak) AS broj_primjeraka - pomoću funkcije agregacije COUNT računam broj primjeraka na svakoj lokaciji te rezultat preimenujem u broj_primjeraka.

- GROUP BY l.id_lokacija, l.odjel, l.kat - podatke grupiram prema identifikatoru lokacije, odjelu i katu kako bih za svaku lokaciju mogao izračunati broj primjeraka.
- CREATE OR REPLACE VIEW fond_po_odjelu - pomoću naredbe CREATE OR REPLACE VIEW kreirao sam pregled pod nazivom fond_po_odjelu koji služi za daljnju analizu knjižničkog fonda po odjelima.

8.4.4. Upit 2: Analiza najzastupljenijih odjela knjižničkog fonda

```
SELECT fpo.odjel,
       fpo.kat,
       SUM(fpo.broj_primjeraka) AS ukupan_broj_primjeraka,
       ROUND(AVG(fpo.broj_primjeraka), 2) AS
prosjecan_broj_primjeraka,
       MAX(fpo.broj_primjeraka) AS
najveci_broj_primjeraka_na_lokaciji,
       MIN(fpo.broj_primjeraka) AS
najmanji_broj_primjeraka_na_lokaciji
FROM fond_po_odjelu AS fpo
GROUP BY fpo.odjel, fpo.kat
HAVING SUM(fpo.broj_primjeraka) >
(
    SELECT AVG(broj_primjeraka)
    FROM fond_po_odjelu
)
ORDER BY ukupan_broj_primjeraka DESC,
         prosjecan_broj_primjeraka DESC,
         fpo.odjel, ASC;
```

Opis upita

- FROM fond_po_odjelu AS fpo - koristim pregled fond_po_odjelu jer su u njemu već pripremljeni podaci o broju primjeraka po pojedinim lokacijama i odjelima knjižnice.
- SELECT - u SELECT dijelu biram podatke koje želim prikazati u rezultatu upita, a to su odjel, kat te ukupni, prosječni, najveći i najmanji broj primjeraka.
- SUM(fpo.broj_primjeraka) AS ukupan_broj_primjeraka - pomoću funkcije agregacije SUM računam ukupan broj primjeraka u svakom odjelu te rezultat preimenujem u ukupan_broj_primjeraka.
- ROUND(AVG(fpo.broj_primjeraka), 2) AS prosjecan_broj_primjeraka - pomoću funkcije AVG računam prosječan broj primjeraka po lokacijama unutar pojedinog odjela, a rezultat zaokružujem na dvije decimale pomoću naredbe ROUND.
- MAX(fpo.broj_primjeraka) AS najveci_broj_primjeraka_na_lokaciji - pomoću funkcije MAX dohvaćam najveći broj primjeraka na pojedinoj lokaciji unutar odjela.

- MIN(fpo.broj_primjeraka) AS najmanji_broj_primjeraka_na_lokaciji - pomoću funkcije MIN dohvaćam najmanji broj primjeraka na pojedinoj lokaciji unutar odjela.
- GROUP BY fpo.odjel, fpo.kat - podatke grupiram prema odjelu i katu kako bih za svaki odjel mogao izračunati tražene vrijednosti.
- HAVING SUM(fpo.broj_primjeraka) > (podupit) - koristim HAVING dio upita kako bih izdvojio samo one odjele koji imaju ukupan broj primjeraka veći od prosjeka svih lokacija u knjižnici.
- SELECT AVG(broj_primjeraka) FROM fond_po_odjelu - pomoću podupita i funkcije AVG računam prosječan broj primjeraka svih lokacija unutar knjižnice.
- ORDER BY ukupan_broj_primjeraka DESC, prosjecan_broj_primjeraka DESC, fpo.odjel ASC - rezultate sortiram silazno prema ukupnom i prosječnom broju primjeraka, a zatim uzlazno prema nazivu odjela radi preglednijeg prikaza rezultata.

– rezultat tablica (?) –

8.4.5. Pogled 3: analiza_fonda_po_zanru

-- Analiza žanrova s natprosječnim brojem primjeraka --

U knjižnici su pojedini žanrovi zastupljeni većim brojem knjiga i primjeraka od ostalih, zbog čega zauzimaju više prostora unutar knjižničkog fonda. Zbog toga je potrebno analizirati koji žanrovi imaju natprosječan broj primjeraka u odnosu na ostatak fonda.

Kako bi se omogućila takva analiza, sustav mora omogućiti pregled ukupnog broja knjiga i primjerak po pojedinim žanrovima.

TRAŽENO RJEŠENJE:

naziv_zanra, broj_knjiga, broj_primjeraka, prosjecan_broj_primjeraka_po_knjizi

```
CREATE OR REPLACE VIEW analiza_fonda_po_zanru AS
SELECT  z.id_zanr,
        z.naziv AS naziv_zanra,
        COUNT(DISTINCT k.id_knjiga) AS broj_knjiga,
        COUNT(p.id_primjerak) AS broj_primjeraka
FROM    zanr AS z
LEFT JOIN knjiga_zanr AS kz
        ON z.id_zanr = kz.id_zanr
LEFT JOIN knjiga AS k
        ON kz.id_knjiga = k.id_knjiga
LEFT JOIN primjerak AS p
        ON k.id_knjiga = p.id_knjiga
GROUP BY z.id_zanr, z.naziv;
```

Opis pregleda

- FROM zanr AS z - koristim tablicu zanr kao glavnu tablicu jer želim analizirati zastupljenost pojedinih žanrova unutar knjižničnog fonda.
- LEFT JOIN knjiga_zanr AS kz ON z.id_zanr = kz.id_zanr - tablicu knjiga_zanr povezujem s tablicom zanr preko atributa id_zanr kako bih mogao dohvatiti knjige koje pripadaju pojedinom žanru.
- LEFT JOIN knjiga AS k ON kz.id_knjiga = k.id_knjiga - tablicu knjiga povezujem preko atributa id_knjiga kako bih mogao dohvatiti podatke o knjigama koje pripadaju određenom žanru.
- LEFT JOIN primjerak AS p ON k.id_knjiga = p.id_knjiga - tablicu primjerak povezujem preko atributa id_knjiga kako bih mogao izračunati broj primjeraka knjiga unutar svakog žanra.
- LEFT JOIN koristim kako bi se u rezultatu prikazali i žanrovi koji trenutno nemaju nijednu knjigu ili primjerak.
- SELECT - u SELECT dijelu biram podatke koje želim prikazati u rezultatu pregleda, a to su identifikator žanra, naziv žanra, broj knjiga i broj primjeraka unutar pojedinog žanra.
- z.naziv AS naziv_zanra - atribut naziv iz tablice zanr preimenujem u naziv_zanra kako bi naziv atributa bio jasniji u rezultatu pregleda.
- COUNT(DISTINCT k.id_knjiga) AS broj_knjiga - pomoću funkcije agregacije COUNT i naredbe DISTINCT računam broj različitih knjiga koje pripadaju pojedinom žanru.
- COUNT(p.id_primjerak) AS broj_primjeraka - pomoću funkcije agregacije COUNT računam ukupan broj primjeraka knjiga unutar pojedinog žanra.
- GROUP BY z.id_zanr, z.naziv - podatke grupiram prema identifikatoru i nazivu žanra kako bih za svaki žanr mogao izračunati broj knjiga i primjeraka.
- CREATE OR REPLACE VIEW analiza_fonda_po_zanru - pomoću naredbe CREATE OR REPLACE VIEW kreirao sam pregled pod nazivom analiza_fonda_po_zanru koji služi za daljnju analizu zastupljenosti žanrova unutar knjižničnog fonda.

8.4.6. Upit 3: Analiza Žanrova s natprosječnim brojem primjeraka

```
SELECT  afpz.naziv_zanra,
        afpz.broj_knjiga,
        afpz.broj_primjeraka,
        ROUND(
            afpz.broj_primjeraka * 1.0 / afpz.broj_knjiga, 2
        ) AS prosjecan_broj_primjeraka_po_knjizi
FROM    analiza_fonda_po_zanru AS afpz
WHERE   afpz.broj_primjeraka >
(
    SELECT AVG(afpz2.broj_primjeraka)
    FROM    analiza_fonda_po_zanru AS afpz2
)
AND     afpz.broj_knjiga > 0
ORDER BY afpz.broj_primjeraka DESC,
        prosjecan_broj_primjeraka_po_knjizi DESC,
        afpz.naziv_zanra ASC;
```

Opis upita

- FROM analiza_fonda_po_zanru AS afpz - koristim pregled analiza_fonda_po_zanru jer su u njemu već pripremljeni podaci o broju knjiga i primjeraka po pojedinim žanrovima.
- SELECT - u SELECT dijelu biram podatke koje želim prikazati u rezultatu upita, a to su naziv žanra, broj knjiga, broj primjeraka i prosječan broj primjeraka po knjizi.
- ROUND(afpz.broj_primjeraka * 1.0 / afpz.broj_knjiga, 2) AS prosjecan_broj_primjeraka_po_knjizi - računam prosječan broj primjeraka po jednoj knjizi unutar pojedinog žanra tako da broj primjeraka podijelim s brojem knjiga, a rezultat zaokružujem na dvije decimale pomoću naredbe ROUND.
- WHERE afpz.broj_primjeraka > (podupit) - koristim WHERE dio upita kako bih izdvojio samo one žanrove koji imaju broj primjeraka veći od prosjeka svih žanrova u knjižnici.
- SELECT AVG(afpz2.broj_primjeraka) FROM analiza_fonda_po_zanru AS afpz2 - pomoću podupita i funkcije AVG računam prosječan broj primjeraka svih žanrova unutar knjižničkog fonda.
- AND afpz.broj_knjiga > 0 - koristim dodatni uvjet kako bih isključio žanrove koji nemaju nijednu knjigu i izbjegao dijeljenje s nulom prilikom izračuna prosječnog broja primjeraka po knjizi.
- ORDER BY afpz.broj_primjeraka DESC, prosjecan_broj_primjeraka_po_knjizi DESC, afpz.naziv_zanra ASC - rezultate sortiram silazno prema broju

primjeraka i prosječnom broju primjeraka po knjizi, a zatim uzlazno prema nazivu žanra radi preglednijeg prikaza rezultata.